

OPEN TELEMETRY В ДЕЙСТВИИ КВЕСТ ПО ВЫСОКОНАГРУЖЕННЫМ СИСТЕМАМ

Специалист по OpenTelemetry в реальном проекте

Модуль 1 - Введение в OpenTelemetry

Урок 1 - История и задачи OpenTelemetry

Шаг 01:

Миссия 1

Юный разработчик слышит о загадочном проекте OpenTelemetry. Какова его главная задача?

Подсказка: Подумай, зачем программисту нужны метрики, логи и трейсы.

Ответ:

- **Собирать и объединять данные наблюдаемости (метрики, логи, трейсы)**
- Заменять Python на более быстрый язык
- Создавать красивые графики интерфейсов
- Хранить данные в базе вместо программиста

Шаг 02:

Миссия 2

Разработчик хочет понять, чем OpenTelemetry отличается от простых логов `print()`. В чём сила?

Подсказка: Подумай о масштабе: от локального эксперимента к продакшену.

Ответ:

- **OpenTelemetry позволяет собирать данные из разных сервисов в едином формате**
- `print()` автоматически отправляет данные в Grafana

- OpenTelemetry заменяет базу данных
- print() подходит для больших распределённых систем

🔮 Шаг 03:

Миссия 3

Герой запускает приложение и хочет знать, что происходит внутри. Какую часть наблюдаемости он подключает первой?

Подсказка: Начни с самого очевидного инструмента, который есть в каждом коде.

Ответ:

- **Логи** — чтобы видеть события программы
- **Метрики** — чтобы сразу видеть бизнес-прибыль
- **Трейсы** — без контекста сервисов они мало полезны
- SQL-запросы к случайной базе

🔮 Шаг 04:

Миссия 4

Архитектор объясняет команде: OpenTelemetry — это не отдельный продукт, а...

Подсказка: Подумай про слово «стандарт».

Ответ:

- **Набор стандартов и инструментов для наблюдаемости**
- Приватная платформа одной компании
- Только библиотека для Python
- Фреймворк для разработки игр

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставь ключевой компонент OpenTelemetry с его назначением.

Компоненты:

Ответ:

- SDK
- Collector
- Экспортеры
- API

Назначения:

1. Собирают данные и передают их в сторонние системы
2. Инструменты для разработчика, чтобы писать код наблюдаемости
3. Приём, обработка и пересылка данных в нужные хранилища
4. Определяет интерфейсы и стандарты работы

Подсказка: Вспомни: API — «что делать», SDK — «как писать», Collector — «куда отправить», Экспортеры — «в какую систему передать».

- **SDK** — Инструменты для разработчика, чтобы писать код наблюдаемости
- **Collector** — Приём, обработка и пересылка данных в нужные хранилища
- **Экспортеры** — Собирают данные и передают их в сторонние системы
- **API** — Определяет интерфейсы и стандарты работы

Шаг 06:

Миссия 6

Формулировка: Сопоставь исторические проекты с их вкладом в появление OpenTelemetry.

Проекты:

Ответ:

- OpenTracing
- OpenCensus
- OpenTelemetry
- Cloud Native Computing Foundation (CNCF)

Вклад:

1. Инициатива, объединившая разные стандарты в один
2. Ранняя попытка унифицировать сбор метрик
3. Создание стандартов для распределённого трейсинга
4. Организация, под крылом которой развивается проект

Подсказка: Два проекта стали «родителями», CNCF дала крышу, а OpenTelemetry объединил усилия.

- **OpenTracing** — Создание стандартов для распределённого трейсинга
- **OpenCensus** — Ранняя попытка унифицировать сбор метрик
- **OpenTelemetry** — Инициатива, объединившая разные стандарты в один
- **Cloud Native Computing Foundation (CNCF)** — Организация, под крылом которой развивается проект

Шаг 07:

Миссия 8

Формулировка: Расставь шаги истории OpenTelemetry в правильном порядке.

1. Появление OpenCensus и OpenTracing
2. Объединение проектов в OpenTelemetry
3. Передача проекта под управление CNCF
4. Активное использование OpenTelemetry в продакшен-системах

Подсказка: Сначала были «родители», потом союз, потом поддержка сообщества, и лишь затем широкое применение.

Ответ:

- Появление OpenCensus и OpenTracing
- Объединение проектов в OpenTelemetry
- Передача проекта под управление CNCF
- Активное использование OpenTelemetry в продакшен-системах

Шаг 08:

Миссия 9

Формулировка: Расставь действия программиста при подключении OpenTelemetry в Python по порядку.

1. Добавить экспортеры для отправки данных
2. Установить библиотеку OpenTelemetry

3. Инструментировать код с помощью API
4. Запустить Collector или систему-назначение

Подсказка: Сначала подключаем библиотеку, потом пишем код, затем настраиваем каналы передачи, и лишь после запускаем обработку.

Ответ:

- Установить библиотеку OpenTelemetry
- Инструментировать код с помощью API
- Добавить экспортеры для отправки данных
- Запустить Collector или систему-назначение

Шаг 09:

Миссия 10

Формулировка: Расставь шаги обработки данных в OpenTelemetry Collector.

1. Обработка (например, фильтрация или агрегация)
2. Экспорт в системы визуализации
3. Получение данных от приложений
4. Очистка и нормализация формата

Подсказка: Думай как о фабрике: сначала сбор сырья, потом очистка, затем обработка и только потом доставка.

Ответ:

- Получение данных от приложений
- Очистка и нормализация формата
- Обработка (например, фильтрация или агрегация)
- Экспорт в системы визуализации

Урок 2 - Основные концепции Trace, Span, Context, Tracer, TracerProvider

Шаг 01:

Миссия 1

Программист запускает сервис и хочет понять, что происходит с запросами. Что в OpenTelemetry отвечает за полное путешествие запроса через систему?

Подсказка: Подумай о маршруте от начала до конца — как след от ракеты в небе.

Ответ:

- **Trace** — полный путь запроса через систему
- **Span** — отдельный участок работы
- **Context** — контейнер для передачи данных
- **Tracer** — инструмент для создания Spans

Шаг 02:

Миссия 2

Ты ловишь баг в сервисе и смотришь на кусочек работы внутри запроса. Какой объект OpenTelemetry описывает именно эту часть?

Подсказка: Представь Trace как книгу, а это — отдельная страница.

Ответ:

- **Span** — отрезок выполнения внутри Trace

- **Trace** — вся история запроса
- **Context** — данные, передаваемые между вызовами
- **TracerProvider** — менеджер для создания Tracers

🔗 Шаг 03:

Миссия 3

У тебя есть цепочка функций, и нужно передать данные о запросе от одной к другой. Какой элемент OpenTelemetry отвечает за это?

Подсказка: Вспомни чемоданчик, в котором передают все важные вещи.

Ответ:

- **Context** — переносит информацию между функциями
- **Trace** — хранит все Spans
- **Span** — измеряет время работы функции
- **Tracer** — инструмент создания Spans

🔗 Шаг 04:

Миссия 4

Ты программист, и тебе нужно создать новые Spans. Какой инструмент OpenTelemetry ты будешь использовать?

Подсказка: Представь себе карандаш, которым рисуют новые события внутри Trace.

Ответ:

- **Tracer** — создаёт новые Spans
- **TracerProvider** — только управляет Tracers
- **Trace** — результат работы Spans
- **Context** — перенос данных

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставь понятие с его определением.

Понятия:

Ответ:

- Trace
- Span
- Context
- Tracer

Определения:

1. Описывает весь путь запроса через систему
2. Отдельный шаг или участок внутри запроса
3. Передаёт данные о текущем запросе между функциями
4. Создает и управляет Span-ами

Подсказка: Подумай: Trace = история, Span = шаг, Context = контейнер, Tracer = создатель.

- **Trace** — Описывает весь путь запроса через систему
- **Span** — Отдельный шаг или участок внутри запроса
- **Context** — Передаёт данные о текущем запросе между функциями

- **Tracer** — Создаёт и управляет Span-ами

Шаг 06:

Миссия 6

Формулировка: Сопоставь объект OpenTelemetry с его примером использования в Python.

Объекты:

Ответ:

- TracerProvider
- Tracer
- Span
- Context

Примеры:

1. Создаёт корневой элемент системы трассировки
2. Используется для создания новых Spans в коде
3. Фиксирует выполнение функции process_data()
4. Хранит активный Span при вызове асинхронной функции

Подсказка: TracerProvider управляет Tracer-ами, Tracer рождает Spans, Span фиксирует событие, Context связывает это между вызовами.

- **TracerProvider** — Создаёт корневой элемент системы трассировки
- **Tracer** — Используется для создания новых Spans в коде
- **Span** — Фиксирует выполнение функции process_data()
- **Context** — Хранит активный Span при вызове асинхронной функции

Шаг 07:

Миссия 7

Формулировка: Сопоставь вопрос разработчика с правильным понятием OpenTelemetry.

Вопросы:

Ответ:

- «Как проследить всю цепочку запроса?»
- «Как измерить время работы функции?»
- «Как передать информацию между микросервисами?»
- «Как создать новый участок трассировки?»

Понятия:

1. Span
2. Trace
3. Context
4. Tracer

Подсказка: Trace = вся история, Span = таймер функции, Context = переносчик, Tracer = создатель.

- «Как проследить всю цепочку запроса?» — Trace
- «Как измерить время работы функции?» — Span
- «Как передать информацию между микросервисами?» — Context
- «Как создать новый участок трассировки?» — Tracer

🔧 Шаг 08:

Миссия 8

Формулировка: Расположи шаги трассировки запроса в правильном порядке.

1. Создание Trace
2. Создание корневого Span
3. Создание вложенных Spans для функций
4. Передача Context между вызовами

Подсказка: Сначала появляется история (Trace), потом первый шаг (Span), затем детали (вложенные Spans), и всё это связывается Context.

Ответ:

- Создание Trace
- Создание корневого Span
- Создание вложенных Spans для функций
- Передача Context между вызовами

🔧 Шаг 09:

Миссия 9

Формулировка: Расположи действия программиста при настройке трассировки в Python.

1. Инициализация TracerProvider
2. Получение Tracer
3. Создание Span для операции
4. Закрытие Span

Подсказка: Сначала создаётся поставщик, потом инструмент, затем фиксируется событие, и только в конце оно завершается.

Ответ:

- Инициализация TracerProvider
- Получение Tracer
- Создание Span для операции
- Закрытие Span

🔧 Шаг 10:

Миссия 10

Формулировка: Расположи уровни вложенности в трассировке от самого общего к самому частному.

1. Trace
2. Span
3. Вложенный Span
4. Событие внутри Span

Подсказка: Подумай как о матрёшке: большая кукла — Trace, внутри неё Spans, внутри них — более мелкие Spans и события.

Ответ:

- Trace
- Span

- Вложенный Span
- Событие внутри Span

Урок 3 - Архитектура OTEL SDK, Collector, Backend, Push vs Pull

Шаг 01:

Миссия 1

Ты подключаешь OpenTelemetry к Python-приложению. Какой компонент отвечает за генерацию метрик и трассировок внутри кода?

Подсказка: Подумай о «библиотеке помощнике», встроенной в твой код.

Ответ:

- **SDK** — библиотека для работы внутри приложения
- **Collector** — сервер для приёма данных
- **Backend** — система хранения и визуализации
- **Exporter** — компонент для передачи данных

Шаг 02:

Миссия 2

Тебе нужно централизованно собирать данные от сотен микросервисов. Какой элемент архитектуры OpenTelemetry используется как «почтальон»?

Подсказка: Представь узел, который принимает письма от всех и сортирует их для доставки.

Ответ:

- SDK
- **Collector** — принимает и пересылает данные
- Backend
- Sampler

Шаг 03:

Миссия 3

Ты хочешь анализировать логи и метрики в удобной панели. Какой компонент OpenTelemetry для этого используется?

Подсказка: Подумай о месте, где хранятся и отображаются все отчёты.

Ответ:

- SDK
- Collector
- **Backend** — система хранения и визуализации
- TracerProvider

Шаг 04:

Миссия 4

Ты думаешь, как доставлять метрики: приложение само отправляет их или сервер их опрашивает? Как называются эти два подхода?

Подсказка: Вспомни: Push = сам пишешь письмо и отправляешь, Pull = ждёшь, когда его попросят.

Ответ:

- **Push и Pull**
- Send и Receive
- Request и Response
- Emit и Collect

 Шаг 05:

Миссия 5

Формулировка: Сопоставь компонент OpenTelemetry с его ролью.

Компоненты:

Ответ:

- SDK
- Collector
- Backend
- Exporter

Роли:

1. Встраивается в код приложения
2. Принимает данные и маршрутизирует дальше
3. Хранит и визуализирует данные
4. Отправляет данные из SDK наружу

Подсказка: SDK = в коде, Collector = пересылка, Backend = хранение, Exporter = доставка.

- **SDK** — Встраивается в код приложения
- **Collector** — Принимает данные и маршрутизирует дальше
- **Backend** — Хранит и визуализирует данные
- **Exporter** — Отправляет данные из SDK наружу

 Шаг 06:

Миссия 6

Формулировка: Сопоставь задачу программиста с правильным инструментом.

Задачи:

Ответ:

- «Хочу встроить трассировку прямо в Python-код»
- «Нужно централизованно принимать потоки данных»
- «Хочу строить красивые графики метрик»
- «Нужно передавать данные в Jaeger»

Инструменты:

1. Exporter
2. SDK
3. Backend
4. Collector

Подсказка: Сначала данные рождаются (SDK), потом отправляются (Exporter), собираются (Collector) и в итоге визуализируются (Backend).

- **«Хочу встроить трассировку прямо в Python-код»** — SDK

- «Нужно централизованно принимать потоки данных» — Collector
- «Хочу строить красивые графики метрик» — Backend
- «Нужно передавать данные в Jaeger» — Exporter

🔮 Шаг 07:

Миссия 7

Формулировка: Сопоставь подход (Push/Pull) с его описанием.

Подходы:

Ответ:

- Push
- Pull

Описание:

1. Приложение само отправляет данные наблюдателю
2. Сервер опрашивает приложение и забирает данные

Подсказка: Push = активная доставка, Pull = «спросили — отдал».

- **Push** — Приложение само отправляет данные наблюдателю
- **Pull** — Сервер опрашивает приложение и забирает данные

🔮 Шаг 08:

Миссия 8

Формулировка: Расположи путь данных от кода до визуализации.

1. Backend
2. Collector
3. SDK
4. Exporter

Подсказка: Сначала данные рождаются (SDK), затем передаются (Exporter), собираются (Collector) и только потом попадают в Backend.

Ответ:

- SDK
- Exporter
- Collector
- Backend

🔮 Шаг 09:

Миссия 9

Формулировка: Расположи шаги настройки Collector-a.

1. Настройка экспортёров (куда отправлять)
2. Определение получателей данных (receivers)
3. Запуск Collector-a
4. Конфигурация процессоров (обработка данных)

Подсказка: Сначала нужно понять откуда брать (receivers), потом как обрабатывать (processors), затем куда отправлять (exporters), и только потом запуск.

Ответ:

- Определение получателей данных (receivers)
- Конфигурация процессоров (обработка данных)
- Настройка экспортёров (куда отправлять)
- Запуск Collector-a

 Шаг 10:

Миссия 10

Формулировка: Расположи уровни архитектуры OpenTelemetry от ближнего к коду до конечного анализа.

1. Backend
2. SDK
3. Collector
4. Exporter

Подсказка: Думай как о реке: источник — это код (SDK), потом данные выходят наружу (Exporter), попадают в хаб (Collector), и лишь потом в море анализа (Backend).

Ответ:

- SDK
- Exporter
- Collector
- Backend

 Урок 4 - Обзор стека OpenTelemetry Collector, Prometheus, Grafana, Jaeger

 Шаг 01:

Миссия 1

Вы только что подключили OpenTelemetry к своему Python-приложению. Какой компонент отвечает за сбор и передачу данных в бекенд?

Подсказка: Представьте, что это почтальон, который доставляет телеграммы от всех частей вашего приложения.

Ответ:

- **OpenTelemetry Collector**
- Prometheus
- Grafana
- Jaeger

 Шаг 02:

Миссия 2

Вы хотите визуализировать метрики вашего приложения. Какой инструмент будет лучшим выбором?

Подсказка: Подумайте, кто умеет красиво рисовать графики и дашборды.

Ответ:

- **Grafana**
- OpenTelemetry Collector
- Python logging

- Jaeger

🔮 Шаг 03:

Миссия 3

Для мониторинга латентности запросов вам нужен инструмент трассировки. Какой из них подходит?

Подсказка: Этот инструмент позволяет видеть путь запроса через все сервисы.

Ответ:

- Jaeger
- Prometheus
- Grafana
- Flask

🔮 Шаг 04:

Миссия 4

Сопоставьте компоненты OpenTelemetry с их ролью в системе:

Ответ:

- OpenTelemetry Collector
 - Prometheus
 - Grafana
 - Jaeger
1. Собирает и передает метрики и трассы
 2. Система хранения и запроса метрик
 3. Визуализация и дашборды
 4. Трассировка запросов между сервисами

Подсказка: Подумайте, кто собирает, кто хранит, кто рисует, а кто показывает путь запроса.

- **OpenTelemetry Collector** — Собирает и передает метрики и трассы
- **Prometheus** — Система хранения и запроса метрик
- **Grafana** — Визуализация и дашборды
- **Jaeger** — Трассировка запросов между сервисами

🔮 Шаг 05:

Миссия 5

Сопоставьте типы данных с их примером:

Ответ:

- Метрика
 - Лог
 - Трейс
1. Количество запросов в секунду
 2. Сообщение об ошибке в приложении
 3. Время прохождения запроса через сервисы

Подсказка: Подумайте, что измеряется количественно, что фиксируется текстом, а что показывает путь.

- **Метрика** — Количество запросов в секунду
- **Лог** — Сообщение об ошибке в приложении
- **Трейс** — Время прохождения запроса через сервисы

Шаг 06:

Миссия 6

Сопоставьте действие с соответствующим инструментом:

Ответ:

- Сбор метрик с приложения
- Визуализация графиков
- Анализ трассировки запросов
- Хранение метрик

1. OpenTelemetry Collector
2. Grafana
3. Jaeger
4. Prometheus

Подсказка: Подумайте, кто собирает, кто хранит, кто показывает, а кто расследует путь запроса.

- **Сбор метрик с приложения** — OpenTelemetry Collector
- **Визуализация графиков** — Grafana
- **Анализ трассировки запросов** — Jaeger
- **Хранение метрик** — Prometheus

Шаг 07:

Миссия 7

Вопрос: Расположите последовательность действий при добавлении OpenTelemetry в Python-приложение:

1. Конфигурация и запуск OpenTelemetry Collector
2. Установка библиотек OpenTelemetry в приложение
3. Сбор данных и их экспорт в Collector
4. Анализ и визуализация в Grafana и Jaeger

Подсказка: Сначала устанавливаем, потом настраиваем, потом собираем и, наконец, смотрим результаты.

Ответ:

- Установка библиотек OpenTelemetry в приложение
- Конфигурация и запуск OpenTelemetry Collector
- Сбор данных и их экспорт в Collector
- Анализ и визуализация в Grafana и Jaeger

Шаг 08:

Миссия 8

Вопрос: Расположите шаги при настройке Prometheus для вашего сервиса:

1. Добавление endpoint метрик в конфигурацию Prometheus

2. Запуск Prometheus и проверка target'ов
3. Сбор метрик с приложения
4. Отображение данных в Grafana

Подсказка: Подумайте, что нужно сначала подключить, потом проверить, потом собирать и, наконец, визуализировать.

Ответ:

- Добавление endpoint метрик в конфигурацию Prometheus
- Запуск Prometheus и проверка target'ов
- Сбор метрик с приложения
- Отображение данных в Grafana

Шаг 09:

Миссия 9

Вопрос: Расположите шаги при расследовании задержек в распределённом Python-приложении с OpenTelemetry:

1. Сбор трассировок с сервисов
2. Анализ трассировки в Jaeger
3. Определение узких мест
4. Оптимизация кода и инфраструктуры

Подсказка: Сначала собираем данные, потом изучаем их, выявляем проблемы и устраняем их.

Ответ:

- Сбор трассировок с сервисов
- Анализ трассировки в Jaeger
- Определение узких мест
- Оптимизация кода и инфраструктуры

Шаг 10:

Миссия 10

Вопрос: Расположите этапы построения комплексной системы мониторинга с OpenTelemetry:

1. Определение метрик и трассируемых операций
2. Интеграция OpenTelemetry SDK в приложение
3. Настройка Collector для экспорта данных
4. Сбор и хранение метрик в Prometheus
5. Визуализация данных в Grafana и анализ трасс в Jaeger

Подсказка: Начните с понимания, что вы хотите измерять, затем подключайте инструменты по цепочке до визуализации и анализа.

Ответ:

- Определение метрик и трассируемых операций
- Интеграция OpenTelemetry SDK в приложение
- Настройка Collector для экспорта данных
- Сбор и хранение метрик в Prometheus
- Визуализация данных в Grafana и анализ трасс в Jaeger

Урок 5 - Почему важно Метрики и трассировка для диагностики

Шаг 01:

Миссия 1

Ты запускаешь новый Python-сервис и хочешь видеть, что происходит внутри. Зачем тебе метрики?

Подсказка: Подумай, как быстро понять, что сервис работает правильно или есть проблемы.

Ответ:

- **Чтобы отслеживать производительность и состояние приложения**
- Чтобы автоматически исправлять ошибки
- Чтобы отключать ненужные функции

Шаг 02:

Миссия 2

Что даёт трассировка в высоконагруженном приложении?

Подсказка: Представь, что твой запрос проходит через десятки сервисов. Как понять, где возникла задержка?

Ответ:

- **Позволяет видеть путь запроса и время выполнения каждой операции**
- Автоматически увеличивает скорость приложения
- Сохраняет только ошибки без контекста

Шаг 03:

Миссия 3

Почему сочетание метрик и трассировок полезно для диагностики?

Подсказка: Подумай о том, как мониторить систему и одновременно понимать, что именно замедляет её работу.

Ответ:

- **Метрики показывают общую картину, трассировки** — детали конкретных запросов
- Метрики полностью заменяют трассировки
- **Трассировки показывают только ошибки, метрики** — только скорость сети

Шаг 04:

Миссия 4

Что произойдет, если не использовать метрики и трассировки в высоконагруженном приложении?

Подсказка: Подумай, как сложно будет найти источник проблем без инструментов наблюдения.

Ответ:

- **Будет трудно понять, где происходят сбои и узкие места**
- Приложение будет работать быстрее
- Ошибки исчезнут сами собой

Шаг 05:

Миссия 5

Сопоставьте понятие с его правильным объяснением:

Понятия:

Ответ:

- Метрики
- Трассировка (Trace)
- Span
- Алерт

Объяснения:

1. Измеряемый показатель работы приложения (CPU, память, запросы/сек)
2. Одиночная операция или шаг в запросе
3. Последовательность связанных span для одного запроса
4. Уведомление о проблеме в приложении

Подсказка: Подумай, что измеряет, что отслеживает путь запроса, а что сигнализирует о проблеме.

- **Метрики** — Измеряемый показатель работы приложения (CPU, память, запросы/сек)
- **Span** — Одиночная операция или шаг в запросе
- **Трассировка (Trace)** — Последовательность связанных span для одного запроса
- **Алерт** — Уведомление о проблеме в приложении

Шаг 06:

Миссия 6

Сопоставьте типы метрик с их назначением:

Типы:

Ответ:

- Counter
- Gauge
- Histogram
- Summary

Назначение:

1. Суммирует количество событий за время
2. Показывает текущее значение, которое может меняться вверх/вниз
3. Регистрирует распределение значений для анализа времени или размера
4. Показывает статистики для выборки, например, процентильные значения

Подсказка: Подумай, что считают, что измеряют мгновенное состояние, а что распределение.

- **Counter** — Суммирует количество событий за время
- **Gauge** — Показывает текущее значение, которое может меняться вверх/вниз
- **Histogram** — Регистрирует распределение значений для анализа времени или размера
- **Summary** — Показывает статистики для выборки, например, процентильные значения

Шаг 07:

Миссия 7

Сопоставьте элементы наблюдения с их ролью:

Элементы:

Ответ:

- Span
- Trace
- Metrics Exporter
- Alert Rule

Роли:

1. Отправляет собранные метрики в систему мониторинга
2. Сигнализирует о превышении порогов метрик или ошибок
3. Отдельная операция или шаг запроса
4. Последовательность связанных span для одного запроса

Подсказка: Подумай о том, кто отправляет данные, кто сигнализирует о проблемах, а кто отслеживает путь запроса.

- **Metrics Exporter** — Отправляет собранные метрики в систему мониторинга
- **Alert Rule** — Сигнализирует о превышении порогов метрик или ошибок
- **Span** — Отдельная операция или шаг запроса
- **Trace** — Последовательность связанных span для одного запроса

 Шаг 08:

Миссия 8

Расположите шаги диагностики медленного запроса с помощью OTEL:

1. Собрать метрики производительности
2. Собрать трассировки для конкретного запроса
3. Проанализировать span для выявления узких мест
4. Составить отчёт или настроить алерт

Подсказка: Сначала общие показатели, потом детали, потом выводы и действия.

Ответ:

- Собрать метрики производительности
- Собрать трассировки для конкретного запроса
- Проанализировать span для выявления узких мест
- Составить отчёт или настроить алерт

 Урок 6 - Почему важно Связь микросервисов и распределённые трассы

 Шаг 01:

Миссия 1

Ты запускаешь несколько микросервисов и замечаешь, что запросы иногда тормозят. Зачем нужны распределённые трассы?

Подсказка: Подумай, как понять, какой сервис замедляет обработку запроса.

Ответ:

- **Чтобы отслеживать путь запроса через все сервисы**
- Чтобы автоматически ускорять медленные сервисы
- Чтобы объединять логи всех сервисов в один файл

Шаг 02:

Миссия 2

Что даёт каждому микросервису внедрение span в код?

Подсказка: Подумай о том, как каждая операция в сервисе становится видимой для анализа.

Ответ:

- **Позволяет детально видеть выполнение операций внутри сервиса**
- Автоматически исправляет ошибки сервиса
- Создаёт резервные копии данных для каждого запроса

Шаг 03:

Миссия 3

Почему важно сохранять идентификатор trace при вызове одного микросервиса из другого?

Подсказка: Подумай о том, как связать spans между разными сервисами.

Ответ:

- **Чтобы все spans одного запроса были объединены в единую трассу**
- Чтобы увеличить скорость передачи данных между сервисами
- Чтобы автоматически создавать новые spans при каждом вызове

Шаг 04:

Миссия 4

Что произойдёт, если не передавать trace-id между микросервисами?

Подсказка: Подумай о том, как потом будет сложно понять путь запроса.

Ответ:

- **Невозможно будет собрать полную трассу запроса**
- Приложение будет быстрее работать
- Микросервисы перестанут общаться друг с другом

Шаг 05:

Миссия 5

Сопоставьте элементы распределённой трассы с их назначением:

Элементы:

Ответ:

- Trace
- Span
- Trace ID
- Parent Span ID

Назначение:

1. Уникальный идентификатор всего запроса через сервисы
2. Одиночная операция или шаг внутри сервиса
3. Последовательность связанных span для одного запроса
4. Связывает span с предыдущим шагом запроса

Подсказка: Подумай, что обозначает весь путь запроса, а что отдельный шаг внутри него.

- **Trace** — Последовательность связанных span для одного запроса
- **Span** — Одиночная операция или шаг внутри сервиса
- **Trace ID** — Уникальный идентификатор всего запроса через сервисы
- **Parent Span ID** — Связывает span с предыдущим шагом запроса

🔗 Шаг 06:

Миссия 6

Сопоставьте преимущества распределённых трасс с объяснениями:

Преимущества:

Ответ:

- Выявление узких мест
- Отладка ошибок
- Мониторинг производительности
- Анализ зависимостей сервисов

Объяснения:

1. Позволяет понять, какой сервис или операция тормозит
2. Помогает найти, где произошёл сбой в цепочке вызовов
3. Отслеживает время выполнения и нагрузку на сервис
4. Видно, как сервисы взаимодействуют между собой

Подсказка: Подумай о том, какие вопросы можно ответить только с помощью trace.

- **Выявление узких мест** — Позволяет понять, какой сервис или операция тормозит
- **Отладка ошибок** — Помогает найти, где произошёл сбой в цепочке вызовов
- **Мониторинг производительности** — Отслеживает время выполнения и нагрузку на сервис
- **Анализ зависимостей сервисов** — Видно, как сервисы взаимодействуют между собой

🔗 Шаг 07:

Миссия 7

Сопоставьте элементы мониторинга с их ролью в распределённых системах:

Элементы:

Ответ:

- Span
- Trace
- Metrics
- Alert

Роли:

1. Отдельная операция в сервисе
2. Последовательность связанных операций запроса
3. Общая картина производительности и состояния
4. Сигнал о проблеме или превышении порога

Подсказка: Подумай, что отслеживает путь запроса, что измеряет показатели, а что сигнализирует о проблемах.

- **Span** — Отдельная операция в сервисе

- **Trace** — Последовательность связанных операций запроса
- **Metrics** — Общая картина производительности и состояния
- **Alert** — Сигнал о проблеме или превышении порога

Шаг 08:

Миссия 8

Расположите шаги для диагностики медленного запроса через несколько микросервисов:

1. Собрать trace и spans для запроса
2. Определить сервисы, которые замедляют обработку
3. Проанализировать spans для выявления узких мест
4. Настроить алерты или отчёты для повторяющихся проблем

Подсказка: Сначала собираем данные, потом анализируем, затем принимаем меры.

Ответ:

- Собрать trace и spans для запроса
- Определить сервисы, которые замедляют обработку
- Проанализировать spans для выявления узких мест
- Настроить алерты или отчёты для повторяющихся проблем

Шаг 09:

Миссия 9

Расположите шаги настройки распределённой трассировки для Python-сервиса:

1. Установить SDK OpenTelemetry и подключить к приложению
2. Создать span вокруг ключевых операций
3. Настроить экспорт trace в наблюдательную систему (Jaeger/Tempo)
4. Передавать trace-id при вызове других сервисов
5. Проверить отображение трасс в наблюдательной системе

Подсказка: Сначала подключаем SDK, затем отмечаем операции, потом экспортируем и проверяем.

Ответ:

- Установить SDK OpenTelemetry и подключить к приложению
- Создать span вокруг ключевых операций
- Настроить экспорт trace в наблюдательную систему (Jaeger/Tempo)
- Передавать trace-id при вызове других сервисов
- Проверить отображение трасс в наблюдательной системе

Шаг 10:

Миссия 10

Расположите шаги для выявления цепочки вызовов, вызывающих ошибки:

1. Собрать trace для проблемного запроса
2. Определить spans, где происходят ошибки
3. Выяснить, какие сервисы участвуют в этих spans
4. Настроить алерты для подобных ошибок в будущем

Подсказка: Сначала данные, потом анализ проблем, затем меры по предотвращению повторений.

Ответ:

- Собрать trace для проблемного запроса
- Определить spans, где происходят ошибки
- Выяснить, какие сервисы участвуют в этих spans
- Настроить алерты для подобных ошибок в будущем

Модуль 2 - Базовая интеграция OTEL в Go-приложения

Урок 1 - Установка и подключение SDK и Exporter

Шаг 01:

Миссия 1

Ты решил интегрировать OpenTelemetry в свой Go-сервис. С чего лучше начать?

Подсказка: Подумай, без чего невозможно собирать данные о трассировке.

Ответ:

- **Установить SDK OpenTelemetry для Go**
- Настроить Jaeger Collector без SDK
- Добавить только экспорт метрик без SDK

Шаг 02:

Миссия 2

Почему важно подключать Exporter после установки SDK?

Подсказка: Подумай, куда должны уходить собранные данные о трассах и метриках.

Ответ:

- **Exporter отправляет данные в наблюдательную систему (Jaeger, Tempo)**
- Exporter создаёт новые spans в приложении
- Exporter ускоряет работу SDK

Шаг 03:

Миссия 3

Что произойдёт, если в Go-приложении не инициализировать SDK перед созданием span?

Подсказка: Подумай, где будет храниться информация о span.

Ответ:

- **Spans не будут собираться, trace будет пустым**
- Приложение автоматически создаст SDK
- Exporter начнёт отправлять пустые данные

Шаг 04:

Миссия 4

Ты установил SDK и Exporter, но spans не доходят до Jaeger. Что нужно проверить первым?

Подсказка: Подумай о том, как данные уходят от приложения к Collector.

Ответ:

- **Правильность конфигурации Exporter и адрес Collector**
- Версию Go SDK

- Количество создаваемых spans

Шаг 05:

Миссия 5

Сопоставьте компоненты OpenTelemetry с их функцией:

Компоненты:

Ответ:

- SDK
- Exporter
- Collector
- Span

Функции:

1. Отправляет собранные данные в систему наблюдения
2. Сбор и создание метрик и трасс
3. Промежуточный сервер, который агрегирует данные и пересылает их
4. Одиночная операция в приложении

Подсказка: Подумай, что делает SDK, что Exporter, а что Span.

- **SDK** — Сбор и создание метрик и трасс
- **Exporter** — Отправляет собранные данные в систему наблюдения
- **Collector** — Промежуточный сервер, который агрегирует данные и пересылает их
- **Span** — Одиночная операция в приложении

Шаг 06:

Миссия 6

Сопоставьте типы Exporter с примерами:

Типы:

Ответ:

- OTLP Exporter
- Jaeger Exporter
- Tempo Exporter
- Console Exporter

Примеры:

1. Отправляет данные на Jaeger UI для просмотра трасс
2. Выводит span в консоль для отладки
3. Отправляет данные через OTLP протокол в Collector
4. Отправляет данные в Grafana Tempo для распределённого trace

Подсказка: Подумай, где отображается trace и какой протокол используется для передачи данных.

- **OTLP Exporter** — Отправляет данные через OTLP протокол в Collector
- **Jaeger Exporter** — Отправляет данные на Jaeger UI для просмотра трасс
- **Tempo Exporter** — Отправляет данные в Grafana Tempo для распределённого trace
- **Console Exporter** — Выводит span в консоль для отладки

Шаг 07:

Миссия 7

Сопоставьте шаги и цели инициализации OpenTelemetry SDK в Go:

Шаги:

Ответ:

- Установка пакетов SDK
- Создание TracerProvider
- Настройка Exporter
- Регистрация глобального Tracer

Цели:

1. Обеспечивает доступ к tracer для приложения
2. Позволяет собирать и хранить данные о spans
3. Устанавливает направление, куда отправлять данные
4. Добавляет SDK в проект и подключает зависимости

Подсказка: Подумай, какой шаг отвечает за регистрацию в коде, а какой за установку зависимостей.

- **Установка пакетов SDK** — Добавляет SDK в проект и подключает зависимости
- **Создание TracerProvider** — Позволяет собирать и хранить данные о spans
- **Настройка Exporter** — Устанавливает направление, куда отправлять данные
- **Регистрация глобального Tracer** — Обеспечивает доступ к tracer для приложения

Урок 2 - Настройка TracerProvider

Шаг 01:

Миссия 1

Ты запускаешь Go-сервис и хочешь собирать распределённые трассы. С чего начать настройку TracerProvider?

Подсказка: Подумай, какой компонент создаёт spans в приложении.

Ответ:

- **Создать экземпляр TracerProvider**
- Настроить Exporter перед SDK
- Установить Collector без SDK

Шаг 02:

Миссия 2

Зачем нужно регистрировать глобальный Tracer после создания TracerProvider?

Подсказка: Подумай, как остальные части приложения узнают, какой Tracer использовать.

Ответ:

- **Чтобы spans создавались через один глобальный Tracer**
- Чтобы Collector автоматически подключился
- Чтобы Exporter стал необязательным

Шаг 03:

Миссия 3

Что произойдёт, если TracerProvider не настроен с Exporter?

Подсказка: Подумай, куда уходят собранные spans.

Ответ:

- **Spans будут собираться локально, но не отправляться в систему наблюдения**
- Приложение автоматически создаст Exporter
- Ничего не произойдёт, все данные будут отправляться

🔗 Шаг 04:

Миссия 4

Ты хочешь использовать несколько Exporter одновременно (Jaeger и Console). Как настроить TracerProvider?

Подсказка: Подумай, как передать несколько получателей данных в SDK.

Ответ:

- **Настроить TracerProvider с несколькими SpanProcessor, каждый для своего Exporter**
- Создать два TracerProvider, каждый с одним Exporter
- Невозможно использовать несколько Exporter одновременно

🔗 Шаг 05:

Миссия 5

Сопоставьте элементы TracerProvider с их функцией:

Элементы:

Ответ:

- TracerProvider
- SpanProcessor
- Exporter
- Tracer

Функции:

1. Создаёт spans внутри приложения
2. Обработывает и передаёт spans от TracerProvider к Exporter
3. Определяет, куда уходят spans (Jaeger, Console, Tempo)
4. Обеспечивает сбор данных о трассах и метриках

Подсказка: Подумай, что создаёт spans, а что их отправляет.

- **TracerProvider** — Обеспечивает сбор данных о трассах и метриках
- **SpanProcessor** — Обработывает и передаёт spans от TracerProvider к Exporter
- **Exporter** — Определяет, куда уходят spans (Jaeger, Console, Tempo)
- **Tracer** — Создаёт spans внутри приложения

🔗 Шаг 06:

Миссия 6

Сопоставьте типы SpanProcessor с их назначением:

Типы:

Ответ:

- SimpleSpanProcessor
- BatchSpanProcessor

Назначение:

1. Отправляет каждый span сразу после создания, для быстрой отладки
2. Группирует spans и отправляет пакетами, повышая производительность

Подсказка: Подумай, когда важна скорость, а когда — эффективность.

- **SimpleSpanProcessor** — Отправляет каждый span сразу после создания, для быстрой отладки
- **BatchSpanProcessor** — Группирует spans и отправляет пакетами, повышая производительность

 Шаг 07:

Миссия 7

Сопоставьте шаги настройки TracerProvider с их действиями:

Шаги:

Ответ:

- Создание TracerProvider
- Добавление SpanProcessor
- Настройка Exporter
- Регистрация глобального Tracer

Действия:

1. Определяет, куда будут отправляться spans
2. Обрабатывает и передаёт spans к Exporter
3. Создаёт TracerProvider для управления сбором данных
4. Делает Tracer доступным для всего приложения

Подсказка: Думай о логическом порядке: сначала создаём, потом настраиваем, потом регистрируем.

- **Создание TracerProvider** — Создаёт TracerProvider для управления сбором данных
- **Добавление SpanProcessor** — Обрабатывает и передаёт spans к Exporter
- **Настройка Exporter** — Определяет, куда будут отправляться spans
- **Регистрация глобального Tracer** — Делает Tracer доступным для всего приложения

 Шаг 08:

Миссия 8

Расположите шаги настройки TracerProvider в правильном порядке:

1. Создать TracerProvider
2. Настроить Exporter
3. Добавить SpanProcessor
4. Регистрация глобального Tracer
5. Создать spans в коде приложения

Подсказка: Сначала создаём провайдер, затем настраиваем, потом регистрируем, и только потом начинаем собирать spans.

Ответ:

- Создать TracerProvider
- Настроить Exporter
- Добавить SpanProcessor
- Регистрация глобального Tracer
- Создать spans в коде приложения

 Шаг 09:**Миссия 9**

Расположите шаги для настройки нескольких Exporter через BatchSpanProcessor:

1. Создать TracerProvider
2. Настроить Jaeger Exporter
3. Настроить Console Exporter
4. Добавить оба Exporter в BatchSpanProcessor
5. Регистрация глобального Tracer
6. Создание span в приложении

Подсказка: Сначала создаём провайдер и экспортёры, затем объединяем их в процессор и регистрируем Tracer.

Ответ:

- Создать TracerProvider
- Настроить Jaeger Exporter
- Настроить Console Exporter
- Добавить оба Exporter в BatchSpanProcessor
- Регистрация глобального Tracer
- Создание span в приложении

 Шаг 10:**Миссия 10**

Расположите этапы инициализации TracerProvider для производственного Go-приложения:

1. Установка зависимостей SDK
2. Создание TracerProvider
3. Настройка BatchSpanProcessor с Exporter
4. Регистрация глобального Tracer
5. Создание и отправка spans в Collector

Подсказка: Думай по логике: сначала установка, потом создание, настройка процессоров, регистрация, и только потом отправка spans.

Ответ:

- Установка зависимостей SDK
- Создание TracerProvider
- Настройка BatchSpanProcessor с Exporter
- Регистрация глобального Tracer
- Создание и отправка spans в Collector

Урок 3 - Создание spans и атрибутов

Шаг 01:

Миссия 1

Ты запускаешь Go-сервис и хочешь отслеживать операции. Что является первым шагом для создания span?

Подсказка: Подумай, какой объект создаёт и управляет spans.

Ответ:

- **Вызвать Tracer и начать span**
- Сразу настроить Exporter
- Создать глобальный Collector без spans

Шаг 02:

Миссия 2

Для чего нужны атрибуты в span?

Подсказка: Подумай, какая информация помогает понять контекст события.

Ответ:

- **Чтобы добавить ключевую информацию о span**
- Чтобы ускорить обработку данных в Exporter
- Чтобы отключить трассировку для этого span

Шаг 03:

Миссия 3

Что происходит, если забыть завершить span вызовом End()?

Подсказка: Подумай, как Collector узнаёт, что span завершён.

Ответ:

- **Span останется открытым и не будет отправлен правильно**
- Span автоматически завершится через 1 секунду
- Ничего не произойдёт, Exporter соберёт его как обычно

Шаг 04:

Миссия 4

Ты хочешь добавить контекст о пользователе в span. Как это лучше сделать?

Подсказка: Подумай, что позволит увидеть дополнительную информацию в системе наблюдения.

Ответ:

- **Добавить атрибуты с ключами user.id и user.role**
- Создать новый TracerProvider для каждого пользователя
- Добавить span без атрибутов

Шаг 05:

Миссия 5

Сопоставьте типы данных span с их назначением:

Ответ:

- Атрибуты
 - События (Events)
 - Статусы (Status)
 - Идентификаторы (IDs)
1. Содержат ключевые значения, описывающие span
 2. Фиксируют важные моменты во времени внутри span
 3. Показывают успешность или ошибку операции
 4. Уникально идентифицируют span и trace

Подсказка: Подумай, что добавляет информацию, а что просто идентифицирует span.

- **Атрибуты** — Содержат ключевые значения, описывающие span
- **События (Events)** — Фиксируют важные моменты во времени внутри span
- **Статусы (Status)** — Показывают успешность или ошибку операции
- **Идентификаторы (IDs)** — Уникально идентифицируют span и trace

 Шаг 06:

Миссия 6

Сопоставьте методы Tracer с их функциями:

Ответ:

- Start
 - End
 - SetAttributes
 - AddEvent
1. Начинает новый span
 2. Заканчивает span и отправляет его через SpanProcessor
 3. Добавляет ключ-значение к span
 4. Фиксирует событие внутри span

Подсказка: Подумай, какие действия соответствуют началу, завершению и наполнению span данными.

- **Start** — Начинает новый span
- **End** — Заканчивает span и отправляет его через SpanProcessor
- **SetAttributes** — Добавляет ключ-значение к span
- **AddEvent** — Фиксирует событие внутри span

 Шаг 07:

Миссия 7

Сопоставьте правильный тип данных span с примером:

Ответ:

- Атрибут
 - Событие
 - Статус
1. "user_id=123"
 2. "DB query started"

3. "ОК"

Подсказка: Подумай, что описывает контекст, что фиксирует событие, а что показывает результат.

- **Атрибут** — "user_id=123"
- **Событие** — "DB query started"
- **Статус** — "ОК"

Шаг 08:

Миссия 8

Расположите шаги создания span с атрибутами в правильном порядке:

1. Вызвать Tracer.Start для создания span
2. Добавить атрибуты через SetAttributes
3. Добавить события через AddEvent
4. Закончить span вызовом End

Подсказка: Сначала создаём span, потом добавляем информацию, затем фиксируем события и только потом завершаем span.

Ответ:

- Вызвать Tracer.Start для создания span
- Добавить атрибуты через SetAttributes
- Добавить события через AddEvent
- Закончить span вызовом End

Урок 4 - Middleware для HTTP (с примерами Recovery и логирования)

Шаг 01:

Миссия 1

Ты только что подключил OpenTelemetry к своему Go-сервису. Какой первый шаг для интеграции middleware в HTTP-сервер?

Подсказка: Подумай, что middleware — это прослойка между сервером и обработчиком запроса.

Ответ:

- **Добавить middleware в цепочку HTTP-обработчиков**
- Игнорировать middleware и писать все логи вручную
- Переписать весь сервер на Python

Шаг 02:

Миссия 2

Какой middleware полезен для автоматического логирования всех HTTP-запросов?

Подсказка: Подумай о middleware, которое «смотрит» каждый запрос и записывает его.

Ответ:

- **Logging middleware**
- Recovery middleware
- Authentication middleware

🔮 Шаг 03:

Миссия 3

Что делает middleware Recovery в Go HTTP-сервере?

Подсказка: Представь, что сервер неожиданно падает из-за ошибки — что должно произойти?

Ответ:

- **Перехватывает панику и возвращает корректный ответ клиенту**
- Игнорирует ошибку и завершает сервер
- Отправляет запросы на другой сервер

🔮 Шаг 04:

Миссия 4

Для чего полезно сочетание Logging и Recovery middleware в высоконагруженных Go-приложениях?

Подсказка: Подумай о сочетании наблюдения за запросами и безопасного восстановления после ошибок.

Ответ:

- **Для отлова ошибок и записи деталей запросов в логи**
- Для ускорения выполнения запросов
- Для автоматического масштабирования сервиса

🔮 Шаг 05:

Миссия 5

Сопоставьте тип middleware с его основной функцией:

Типы:

Ответ:

- Logging
- Recovery
- Authentication
- Metrics

Функции:

1. Перехватывает панику и предотвращает падение сервера
2. Собирает показатели производительности и количество запросов
3. Проверяет, что пользователь имеет доступ к ресурсу
4. Записывает информацию о каждом HTTP-запросе

Подсказка: Подумай, кто что «делает» в цепочке обработки запроса.

- **Logging** — Записывает информацию о каждом HTTP-запросе
- **Recovery** — Перехватывает панику и предотвращает падение сервера
- **Authentication** — Проверяет, что пользователь имеет доступ к ресурсу
- **Metrics** — Собирает показатели производительности и количество запросов

🔮 Шаг 06:

Миссия 6

Сопоставьте подходы интеграции OTEL middleware с их эффектом:

Подходы:

Ответ:

- Оборачивание каждого хендлера
- Использование глобальной цепочки middleware
- Применение middleware только к критическим маршрутам
- Комбинация logging и metrics

Эффекты:

1. Позволяет собирать данные только о важных запросах
2. Обеспечивает единообразие трассировки для всех запросов
3. Позволяет детально контролировать обработку каждого обработчика
4. Собирает и логи, и метрики одновременно

Подсказка: Думай о масштабах применения и комбинировании функций.

- **Оборачивание каждого хендлера** — Позволяет детально контролировать обработку каждого обработчика
- **Использование глобальной цепочки middleware** — Обеспечивает единообразие трассировки для всех запросов
- **Применение middleware только к критическим маршрутам** — Позволяет собирать данные только о важных запросах
- **Комбинация logging и metrics** — Собирает и логи, и метрики одновременно

Шаг 07:

Миссия 7

Сопоставьте тип трассировки OTEL с его назначением:

Типы:

Ответ:

- Span
- Trace
- Exporter
- Context

Назначение:

1. Передаёт данные о выполнении запроса между middleware и обработчиками
2. Представляет отдельную операцию или блок кода для мониторинга
3. Собирает все связанные span в единый набор для анализа
4. Отправляет собранные данные в систему наблюдения

Подсказка: Подумай о том, как строится цепочка обработки запроса и как собираются метрики.

- **Span** — Представляет отдельную операцию или блок кода для мониторинга
- **Trace** — Собирает все связанные span в единый набор для анализа
- **Exporter** — Отправляет собранные данные в систему наблюдения
- **Context** — Передаёт данные о выполнении запроса между middleware и обработчиками

Шаг 08:

Миссия 8

Расположите шаги интеграции OTEL middleware в правильном порядке:

1. Настройка экспортеров (Prometheus, Jaeger и т.д.)
2. Добавление middleware в цепочку HTTP-хендлеров
3. Создание и инициализация трассера
4. Тестирование и проверка корректной работы

Подсказка: Сначала создаём инструменты, потом добавляем их в сервер, а в конце проверяем.

Ответ:

- Создание и инициализация трассера
- Настройка экспортеров (Prometheus, Jaeger и т.д.)
- Добавление middleware в цепочку HTTP-хендлеров
- Тестирование и проверка корректной работы

Шаг 09:

Миссия 9

Расположите действия при обработке запроса через middleware в правильном порядке:

1. Recovery проверяет панику
2. Logging записывает запрос
3. Основной хендлер выполняет бизнес-логику
4. Метрики собираются и отправляются экспортеру

Подсказка: Подумай о порядке «наблюдение → выполнение → сбор статистики».

Ответ:

- Logging записывает запрос
- Recovery проверяет панику
- Основной хендлер выполняет бизнес-логику
- Метрики собираются и отправляются экспортеру

Шаг 10:

Миссия 10

Расположите этапы комплексной настройки OTEL в высоконагруженном Go-сервисе:

1. Добавление middleware Logging и Recovery
2. Настройка трассера и экспортеров
3. Определение критических маршрутов для мониторинга
4. Тестирование и проверка всех метрик и логов

Подсказка: Сначала подготавливаем инструменты, затем применяем к нужным маршрутам, и только потом проверяем.

Ответ:

- Настройка трассера и экспортеров
- Добавление middleware Logging и Recovery
- Определение критических маршрутов для мониторинга
- Тестирование и проверка всех метрик и логов

Урок 5 - Отправка spans в Collector Jaeger Tempo

Шаг 01:

Миссия 1

Ты только что подключил OpenTelemetry к Go-приложению. Какой первый шаг для отправки span в Collector?

Подсказка: Подумай, что сначала нужно определить, куда будут уходить данные.

Ответ:

- **Настроить экспортёр (Collector, Jaeger, Tempo)**
- Игнорировать Collector и писать все span локально
- Сразу отправлять span в базу данных

Шаг 02:

Миссия 2

Какой формат экспорта чаще всего используется для Jaeger и Tempo?

Подсказка: Подумай, какой протокол понимают эти системы.

Ответ:

- **OTLP/HTTP или Thrift**
- CSV или JSON локально
- SOAP или XML

Шаг 03:

Миссия 3

Что нужно сделать, чтобы span отправлялся из Go-приложения в Collector асинхронно и без блокировки сервера?

Подсказка: Подумай о буферизации и фоновой отправке.

Ответ:

- **Использовать батчинг и асинхронный экспортер**
- Отправлять каждый span сразу при его создании
- Сохранять span только в локальный файл

Шаг 04:

Миссия 4

Какая настройка помогает избежать потери span при падении приложения?

Подсказка: Подумай о том, что экспортер может потерять данные, если приложение завершилось внезапно.

Ответ:

- **Настройка сохранения и повторной отправки при сбое**
- Игнорирование ошибок экспорта
- Отправка span только при завершении процесса

Шаг 05:

Миссия 5

Сопоставьте тип экспорта OTEL с его назначением:

Типы:

Ответ:

- Collector
- Jaeger
- Tempo
- OTLP

Назначение:

1. Собирать трассировки и пересылать их в другие системы
2. Система визуализации распределённых трассировок, удобная для поиска span
3. Система хранения временных трассировок, ориентированная на масштабируемость
4. Протокол передачи span между приложением и Collector

Подсказка: Подумай, кто что «делает» в цепочке передачи span.

- **Collector** — Собирать трассировки и пересылать их в другие системы
- **Jaeger** — Система визуализации распределённых трассировок, удобная для поиска span
- **Tempo** — Система хранения временных трассировок, ориентированная на масштабируемость
- **OTLP** — Протокол передачи span между приложением и Collector

Шаг 06:

Миссия 6

Сопоставьте понятия OTEL export pipeline с их функциями:

Понятия:

Ответ:

- Batch Exporter
- Simple Exporter
- Span Processor
- Exporter

Функции:

1. Собирает span и отправляет сразу после создания
2. Собирает span в пакет и отправляет периодически
3. Обработывает span перед отправкой (например, добавляет атрибуты)
4. Отправляет span в систему хранения или визуализации

Подсказка: Подумай о разнице между немедленной отправкой и пакетной отправкой.

- **Simple Exporter** — Собирает span и отправляет сразу после создания
- **Batch Exporter** — Собирает span в пакет и отправляет периодически
- **Span Processor** — Обработывает span перед отправкой (например, добавляет атрибуты)
- **Exporter** — Отправляет span в систему хранения или визуализации

Шаг 07:

Миссия 7

Сопоставьте термин с его ролью при отправке span:

Термины:

Ответ:

- Context Propagation
- Span
- Trace
- Exporter Configuration

Роли:

1. Передаёт информацию о span между сервисами
2. Одиночная операция или блок кода для мониторинга
3. Собирает связанные span в единый набор для анализа
4. Задаёт параметры для отправки span в Collector или Jaeger

Подсказка: Подумай, что отвечает за передачу, что за хранение и что за конфигурацию.

- **Context Propagation** — Передаёт информацию о span между сервисами
- **Span** — Одиночная операция или блок кода для мониторинга
- **Trace** — Собирает связанные span в единый набор для анализа
- **Exporter Configuration** — Задаёт параметры для отправки span в Collector или Jaeger

 Шаг 08:

Миссия 8

Расположите шаги отправки span в правильном порядке:

1. Создание span в приложении
2. Обработка span через Span Processor
3. Буферизация span в Batch Exporter
4. Отправка span в Collector/Jaeger/Tempo

Подсказка: Сначала span создаётся, потом обрабатывается, затем собирается в пакет, и только после этого отправляется.

Ответ:

- Создание span в приложении
- Обработка span через Span Processor
- Буферизация span в Batch Exporter
- Отправка span в Collector/Jaeger/Tempo

 Шаг 09:

Миссия 9

Расположите действия по настройке OTEL экспортёра для Jaeger в Go-приложении:

1. Указать URL сервиса Jaeger
2. Выбрать протокол передачи span (OTLP/HTTP)
3. Инициализировать экспортер
4. Добавить экспортер в pipeline обработки span
5. Тестирование и проверка трассировок

Подсказка: Сначала настраиваем соединение, затем инициализируем, подключаем и проверяем.

Ответ:

- Указать URL сервиса Jaeger
- Выбрать протокол передачи span (OTLP/HTTP)

- Инициализировать экспортер
- Добавить экспортер в pipeline обработки span
- Тестирование и проверка трассировок

Шаг 10:

Миссия 10

Расположите этапы комплексной настройки отправки span в высоконагруженном Go-сервисе:

1. Выбор и настройка экспортёра
2. Создание и добавление span processor
3. Добавление middleware и генерация span
4. Буферизация и отправка батчами
5. Проверка трассировок в Jaeger/Tempo

Подсказка: Сначала готовим инструменты, потом создаём span, обрабатываем и отправляем, и только в конце проверяем.

Ответ:

- Выбор и настройка экспортёра
- Создание и добавление span processor
- Добавление middleware и генерация span
- Буферизация и отправка батчами
- Проверка трассировок в Jaeger/Tempo

Модуль 3 - Внутреннее устройство OTEL

Урок 1 - Жизненный цикл span Создание, атрибуты, события, завершение

Шаг 01:

Миссия 1

Ты создаёшь новый span в Go-сервисе. Какой метод используется для его начала?

Подсказка: Подумай, что именно запускает span.

Ответ:

- **Tracer.Start**
- Tracer.End
- Span.SetAttributes

Шаг 02:

Миссия 2

Для чего используются атрибуты span?

Подсказка: Подумай, как можно добавить контекст к операции.

Ответ:

- **Для хранения ключ-значение о span**
- Для завершения span автоматически
- Для запуска нового TracerProvider

🔗 Шаг 03:

Миссия 3

Что произойдет, если забыть вызвать `End()` для `span`?

Подсказка: Подумай, как `Collector` узнаёт, что `span` завершён.

Ответ:

- **Span останется открытым и не будет отправлен**
- `Span` завершится автоматически через секунду
- `Exporter` соберёт `span` без `End`

🔗 Шаг 04:

Миссия 4

Как правильно зафиксировать событие внутри `span`?

Подсказка: Подумай, какой метод используется для событий.

Ответ:

- **AddEvent**
- `SetAttributes`
- `Start`

🔗 Шаг 05:

Миссия 5

Сопоставьте компоненты `span` с их назначением:

Ответ:

- Атрибуты
 - События
 - Статус
 - ID
1. Ключ-значение с контекстом
 2. Фиксация важных моментов во времени
 3. Отображение успешности или ошибки
 4. Уникальная идентификация `span` и `trace`

Подсказка: Подумай, что описывает данные, что фиксирует момент, а что идентифицирует `span`.

- **Атрибуты** — Ключ-значение с контекстом
- **События** — Фиксация важных моментов во времени
- **Статус** — Отображение успешности или ошибки
- **ID** — Уникальная идентификация `span` и `trace`

🔗 Шаг 06:

Миссия 6

Сопоставьте методы `span` с их функциями:

Ответ:

- `Start`
- `End`
- `SetAttributes`

- AddEvent

1. Создает новый span
2. Завершает span и отправляет его
3. Добавляет ключ-значение к span
4. Фиксирует событие внутри span

Подсказка: Сначала создаём span, затем наполняем данными, фиксируем события и завершаем.

- **Start** — Создает новый span
- **End** — Завершает span и отправляет его
- **SetAttributes** — Добавляет ключ-значение к span
- **AddEvent** — Фиксирует событие внутри span

Шаг 07:

Миссия 7

Сопоставьте тип span с примером:

Ответ:

- Атрибут
 - Событие
 - Статус
1. "user_id=42"
 2. "DB query started"
 3. "OK"

Подсказка: Подумай, что добавляет контекст, что фиксирует момент, а что результат операции.

- **Атрибут** — "user_id=42"
- **Событие** — "DB query started"
- **Статус** — "OK"

Шаг 08:

Миссия 8

Расположите шаги жизненного цикла span в правильном порядке:

1. Создать span с Tracer.Start
2. Добавить атрибуты с SetAttributes
3. Добавить события с AddEvent
4. Завершить span вызовом End

Подсказка: Сначала span создается, затем наполняется данными, фиксируются события и завершается.

Ответ:

- Создать span с Tracer.Start
- Добавить атрибуты с SetAttributes
- Добавить события с AddEvent
- Завершить span вызовом End

Шаг 09:

Миссия 9

Расположите действия для отслеживания HTTP-запроса через spans:

1. Создать span "HTTP GET /items"
2. Добавить атрибуты: метод, URL, статус
3. Добавить событие "Request received"
4. Выполнить запрос к серверу
5. Добавить событие "Response sent"
6. Закончить span

Подсказка: Сначала создаём span, затем фиксируем события и атрибуты во время запроса, после чего завершаем span.

Ответ:

- Создать span "HTTP GET /items"
- Добавить атрибуты: метод, URL, статус
- Добавить событие "Request received"
- Выполнить запрос к серверу
- Добавить событие "Response sent"
- Закончить span

Шаг 10:

Миссия 10

Расположите шаги работы span для функции обработки платежа:

1. Start span "ProcessPayment"
2. SetAttributes: user_id, amount
3. AddEvent: "Validation started"
4. Выполнить валидацию и обработку платежа
5. AddEvent: "Payment completed"
6. End span

Подсказка: Логика span: создаём → добавляем контекст → фиксируем события во время процесса → завершаем.

Ответ:

- Start span "ProcessPayment"
- SetAttributes: user_id, amount
- AddEvent: "Validation started"
- Выполнить валидацию и обработку платежа
- AddEvent: "Payment completed"
- End span

Урок 2 - Контекст и наследование родителя (ParentChild spans)

Шаг 01:

Миссия 1

Какой span считается родительским?

Подсказка: Подумай, какой span запускает другие spans.

Ответ:

- **Span, от которого создаются дочерние spans**
- Span без атрибутов

- Span, который завершился первым

🔮 Шаг 02:

Миссия 2

Как дочерний span получает контекст родителя?

Подсказка: Контекст передается при создании span.

Ответ:

- Через Context при вызове **Tracer.Start**
- Через SetAttributes после создания span
- Автоматически без передачи контекста

🔮 Шаг 03:

Миссия 3

Что произойдет, если дочерний span создается без передачи родителя?

Подсказка: Подумай, как система строит деревья spans.

Ответ:

- Span станет корневым, без родителя
- Span наследует случайного родителя
- Span не создастся

🔮 Шаг 04:

Миссия 4

Можно ли создать несколько дочерних spans от одного родителя?

Подсказка: Подумай о параллельных процессах в одном контексте.

Ответ:

- Да, можно создавать несколько дочерних spans от одного родителя
- Нет, каждый родитель может иметь только один дочерний span

🔮 Шаг 05:

Миссия 5

Сопоставьте span с его характеристикой:

Ответ:

- Parent span
 - Child span
 - Root span
1. Span, создающий другие spans
 2. Span, наследующий контекст родителя
 3. Span без родителя, стартующий новый trace

Подсказка: Подумай о дереве spans: кто вершина, кто наследник.

- **Parent span** — Span, создающий другие spans
- **Child span** — Span, наследующий контекст родителя
- **Root span** — Span без родителя, стартующий новый trace

🔗 Шаг 06:

Миссия 6

Сопоставьте действия span с их эффектом на наследование контекста:

Ответ:

- Создание span с Context
 - Создание span без Context
 - Завершение родительского span
1. Дочерний span наследует родителя
 2. Span становится корневым
 3. Дочерние spans сохраняют ссылку на родителя

Подсказка: Подумай, что влияет на иерархию spans.

- **Создание span с Context** — Дочерний span наследует родителя
- **Создание span без Context** — Span становится корневым
- **Завершение родительского span** — Дочерние spans сохраняют ссылку на родителя

🔗 Шаг 07:

Миссия 8

Расположите действия при обработке родительского и дочернего span:

1. Start parent span
2. Start child span
3. Добавление атрибутов к child span
4. End child span
5. End parent span

Подсказка: Сначала создаётся родительский span, затем его дети, затем завершаются сначала дети, потом родитель.

Ответ:

- Start parent span
- Start child span
- Добавление атрибутов к child span
- End child span
- End parent span

🔗 Шаг 08:

Миссия 9

Расположите события при обработке HTTP запроса с parent/child spans:

1. Start root span "Request"
2. Start child span "DB Query"
3. AddEvent "Query started"
4. End child span
5. AddEvent "Response sent"
6. End root span

Подсказка: Помни, события привязываются к конкретному span, завершение дочернего span происходит до родителя.

Ответ:

- Start root span "Request"
- Start child span "DB Query"
- AddEvent "Query started"
- End child span
- AddEvent "Response sent"
- End root span

 Шаг 09:

Миссия 10

Расположите шаги для цепочки parent/child spans при обработке платежа:

1. Start root span "ProcessPayment"
2. Start child span "ValidatePayment"
3. End child span "ValidatePayment"
4. Start child span "ChargeCard"
5. End child span "ChargeCard"
6. End root span "ProcessPayment"

Подсказка: Сначала создаётся root, затем дочерние spans последовательно, завершаясь сначала дети, потом root.

Ответ:

- Start root span "ProcessPayment"
- Start child span "ValidatePayment"
- End child span "ValidatePayment"
- Start child span "ChargeCard"
- End child span "ChargeCard"
- End root span "ProcessPayment"

 Урок 3 - TracerProvider и Batch Span Processor

 Шаг 01:

Миссия 1

Что делает TracerProvider в OpenTelemetry?

Подсказка: Подумайте о том, кто создаёт spans для приложения.

Ответ:

- **Предоставляет Tracer для создания spans**
- Обработывает события логирования
- Хранит метрики CPU и памяти

 Шаг 02:

Миссия 2

Для чего нужен BatchSpanProcessor?

Подсказка: Подумайте о том, как spans отправляются эффективно.

Ответ:

- **Буферизация и периодическая отправка spans**
- Создание родительских span

- Сбор метрик из приложения

🔧 Шаг 03:

Миссия 3

Что произойдёт, если не добавить BatchSpanProcessor в TracerProvider?

Подсказка: Подумайте о том, как spans будут отправляться.

Ответ:

- **Каждый span будет отправляться сразу, без буфера**
- Spans будут автоматически удаляться
- TracerProvider не сможет создавать spans

🔧 Шаг 04:

Миссия 4

Какой компонент отвечает за отправку spans к экспортеру?

Подсказка: BatchSpanProcessor управляет периодической отправкой.

Ответ:

- **SpanProcessor**
- TracerProvider
- Sampler

🔧 Шаг 05:

Миссия 5

Сопоставьте понятия с их ролью:

Ответ:

- TracerProvider
 - BatchSpanProcessor
 - Span
1. Создаёт Tracer для генерации spans
 2. Буферизация и отправка spans к экспортеру
 3. Единица трассировки операций

Подсказка: Подумайте о цепочке: кто создаёт, кто обрабатывает, что создаётся.

- **TracerProvider** — Создаёт Tracer для генерации spans
- **BatchSpanProcessor** — Буферизация и отправка spans к экспортеру
- **Span** — Единица трассировки операций

🔧 Шаг 06:

Миссия 6

Сопоставьте методы TracerProvider с их назначением:

Ответ:

- AddSpanProcessor
 - GetTracer
 - Shutdown
1. Добавляет BatchSpanProcessor

2. Возвращает Tracer для создания spans
3. Завершает работу TracerProvider и отправляет остатки spans

Подсказка: Подумайте о действиях с TracerProvider в начале, середине и конце работы.

- **AddSpanProcessor** — Добавляет BatchSpanProcessor
- **GetTracer** — Возвращает Tracer для создания spans
- **Shutdown** — Завершает работу TracerProvider и отправляет остатки spans

Шаг 07:

Миссия 7

Сопоставьте типы SpanProcessor с их особенностями:

Ответ:

- BatchSpanProcessor
 - SimpleSpanProcessor
1. Буферизация и периодическая отправка
 2. Мгновенная отправка каждого span

Подсказка: Один оптимизирован для нагрузки, другой — для простоты.

- **BatchSpanProcessor** — Буферизация и периодическая отправка
- **SimpleSpanProcessor** — Мгновенная отправка каждого span

Шаг 08:

Миссия 8

Расположите шаги настройки TracerProvider с BatchSpanProcessor:

1. Создать TracerProvider
2. Создать BatchSpanProcessor с экспортером
3. Добавить BatchSpanProcessor к TracerProvider
4. Получить Tracer и создавать spans

Подсказка: Думайте, что создаём сначала, что подключаем, а что используем после.

Ответ:

- Создать TracerProvider
- Создать BatchSpanProcessor с экспортером
- Добавить BatchSpanProcessor к TracerProvider
- Получить Tracer и создавать spans

Шаг 09:

Миссия 9

Расположите действия при завершении работы приложения с BatchSpanProcessor:

1. Создание span
2. Буферизация span BatchSpanProcessor
3. Вызов Shutdown TracerProvider
4. Отправка остатка spans экспортеру

Подсказка: Сначала создаются spans, затем они буферизуются, а при завершении приложения отправляются остатки.

Ответ:

- Создание span
- Буферизация span BatchSpanProcessor
- Вызов Shutdown TracerProvider
- Отправка остатка spans экспортеру

 Шаг 10:

Миссия 10

Расположите шаги для настройки TracerProvider с несколькими SpanProcessors:

1. Создать TracerProvider
2. Создать BatchSpanProcessor
3. Создать SimpleSpanProcessor
4. Добавить оба SpanProcessors к TracerProvider
5. Получить Tracer и создавать spans

Подсказка: Создание компонентов сначала, подключение, затем использование Tracer.

Ответ:

- Создать TracerProvider
- Создать BatchSpanProcessor
- Создать SimpleSpanProcessor
- Добавить оба SpanProcessors к TracerProvider
- Получить Tracer и создавать spans

 Урок 4 - Экспорт данных через OTLP Exporter

 Шаг 01:

Миссия 1

Что делает OTLP Exporter в OpenTelemetry?

Подсказка: Подумайте, куда уходят данные трассировки.

Ответ:

- **Отправляет spans и метрики в внешнюю систему**
- Создает новые spans
- Буферизует данные внутри приложения

 Шаг 02:

Миссия 2

Какой протокол использует OTLP Exporter по умолчанию?

Подсказка: Подумайте о современном стандарте передачи телеметрии.

Ответ:

- **gRPC**
- HTTP/1.1
- FTP

 Шаг 03:

Миссия 3

Что произойдёт, если OTLP Exporter недоступен?

Подсказка: Подумайте, как BatchSpanProcessor обрабатывает неудачные попытки отправки.

Ответ:

- **Данные будут буферизоваться до восстановления соединения**
- TracerProvider перестанет работать
- Приложение аварийно завершится

Шаг 04:

Миссия 4

Какая конфигурация OTLP Exporter отвечает за конечный пункт отправки данных?

Подсказка: Подумайте, что указывает адрес сервера для телеметрии.

Ответ:

- **Endpoint**
- Sampler
- TracerName

Шаг 05:

Миссия 5

Сопоставьте параметры OTLP Exporter с их назначением:

Ответ:

- Endpoint
 - Timeout
 - Headers
1. URL сервера для отправки данных
 2. Максимальное время ожидания ответа от сервера
 3. Дополнительные HTTP заголовки для аутентификации

Подсказка: Подумайте, что указывает адрес, что управляет временем, а что — безопасностью.

- **Endpoint** — URL сервера для отправки данных
- **Timeout** — Максимальное время ожидания ответа от сервера
- **Headers** — Дополнительные HTTP заголовки для аутентификации

Шаг 06:

Миссия 6

Сопоставьте типы данных, которые OTLP Exporter может отправлять:

Ответ:

- Spans
 - Метрики
 - Логи
1. Данные о трассировке операций
 2. Количественные показатели приложения
 3. Записи событий приложения

Подсказка: Подумайте о смысле каждого типа данных.

- **Spans** — Данные о трассировке операций
- **Метрики** — Количественные показатели приложения
- **Логи** — Записи событий приложения

Шаг 07:

Миссия 7

Сопоставьте компоненты OpenTelemetry с их ролью при экспорте:

Ответ:

- Tracer
- BatchSpanProcessor
- OTLP Exporter

1. Создает spans для операций
2. Буферизует и отправляет spans
3. Отправляет данные на сервер

Подсказка: Подумайте о цепочке создания и передачи данных.

- **Tracer** — Создает spans для операций
- **BatchSpanProcessor** — Буферизует и отправляет spans
- **OTLP Exporter** — Отправляет данные на сервер

Шаг 08:

Миссия 8

Расположите шаги передачи span через OTLP Exporter:

1. Создание span через Tracer
2. BatchSpanProcessor буферизует span
3. BatchSpanProcessor передаёт span OTLP Exporter
4. OTLP Exporter отправляет span на сервер

Подсказка: Думайте о последовательности: создание → буферизация → передача → отправка.

Ответ:

- Создание span через Tracer
- BatchSpanProcessor буферизует span
- BatchSpanProcessor передаёт span OTLP Exporter
- OTLP Exporter отправляет span на сервер

Шаг 09:

Миссия 9

Расположите действия при конфигурации OTLP Exporter:

1. Создать OTLP Exporter с endpoint
2. Создать BatchSpanProcessor и подключить экспортёр
3. Добавить BatchSpanProcessor к TracerProvider
4. Создать Tracer и начать создавать spans

Подсказка: Сначала создаём экспортёр, потом соединяем и используем Tracer.

Ответ:

- Создать OTLP Exporter с endpoint
- Создать BatchSpanProcessor и подключить экспортер
- Добавить BatchSpanProcessor к TracerProvider
- Создать Tracer и начать создавать spans

 Шаг 10:

Миссия 10

Расположите шаги для безопасной отправки spans через OTLP Exporter:

1. Установить endpoint с TLS
2. Добавить заголовки для аутентификации
3. Создать BatchSpanProcessor и подключить OTLP Exporter
4. Добавить BatchSpanProcessor к TracerProvider
5. Создать Tracer и начать создавать spans

Подсказка: Сначала безопасность, потом соединение и использование Tracer.

Ответ:

- Установить endpoint с TLS
- Добавить заголовки для аутентификации
- Создать BatchSpanProcessor и подключить OTLP Exporter
- Добавить BatchSpanProcessor к TracerProvider
- Создать Tracer и начать создавать spans

 Урок 5 - Как OTEL управляет нагрузкой и памятью

 Шаг 01:

Миссия 1

Что делает BatchSpanProcessor при высокой нагрузке?

Подсказка: Подумайте, зачем нужны буферы и пакетная обработка.

Ответ:

- **Собирает spans в пакеты и отправляет их пакетно**
- Создает новые spans для каждого события
- Сохраняет все spans навсегда в памяти

 Шаг 02:

Миссия 2

Что происходит, если буфер BatchSpanProcessor заполнен?

Подсказка: Подумайте о механизме ограничения памяти.

Ответ:

- **Старые spans могут быть сброшены или отправлены раньше**
- Приложение аварийно завершится
- Буфер продолжит расти без ограничений

 Шаг 03:

Миссия 3

Что делает OTEL, чтобы снизить нагрузку на сеть при экспорте данных?

Подсказка: Подумайте о пакетной отправке и интервалах экспорта.

Ответ:

- **Отправляет spans пакетами через заданные интервалы времени**
- Отправляет каждый span сразу
- Сохраняет все spans в память до завершения работы приложения

🔧 Шаг 04:

Миссия 4

Как OTEL предотвращает переполнение памяти при высоком потоке данных?

Подсказка: Подумайте о лимитах буфера и стратегии отброса данных.

Ответ:

- **Использует лимит буфера и сбрасывает старые spans при необходимости**
- Накапливает все spans без ограничения
- Останавливает приложение при переполнении

🔧 Шаг 05:

Миссия 5

Сопоставьте термин и его роль в управлении нагрузкой OTEL:

Ответ:

- BatchSpanProcessor
 - Буфер
 - Export Interval
 - Max Queue Size
1. Определяет частоту отправки пакетов spans
 2. Хранит временно spans перед отправкой
 3. Управляет размером очереди и предотвращает переполнение памяти
 4. Объединяет spans в пакеты для экспорта

Подсказка: Подумайте о функциях компонента и параметров.

- **BatchSpanProcessor** — Объединяет spans в пакеты для экспорта
- **Буфер** — Хранит временно spans перед отправкой
- **Export Interval** — Определяет частоту отправки пакетов spans
- **Max Queue Size** — Управляет размером очереди и предотвращает переполнение памяти

🔧 Шаг 06:

Миссия 6

Сопоставьте понятия управления памятью OTEL с их объяснением:

Ответ:

- Drop Policy
 - Batch Timeout
 - Queue Overflow
1. Стратегия удаления старых spans при переполнении
 2. Максимальное время ожидания перед отправкой пакета

3. Ситуация, когда буфер превышает лимит

Подсказка: Подумайте о том, как OTEL справляется с большим количеством данных.

- **Drop Policy** — Стратегия удаления старых spans при переполнении
- **Batch Timeout** — Максимальное время ожидания перед отправкой пакета
- **Queue Overflow** — Ситуация, когда буфер превышает лимит

🔧 Шаг 07:

Миссия 7

Сопоставьте компоненты OTEL с их функцией при высокой нагрузке:

Ответ:

- Tracer
 - BatchSpanProcessor
 - OTLP Exporter
1. Создаёт spans для операций приложения
 2. Буферизует и пакетирует spans
 3. Отправляет spans на сервер

Подсказка: Думайте о цепочке создания и отправки spans.

- **Tracer** — Создаёт spans для операций приложения
- **BatchSpanProcessor** — Буферизует и пакетирует spans
- **OTLP Exporter** — Отправляет spans на сервер

🔧 Шаг 08:

Миссия 8

Расположите шаги обработки span при высокой нагрузке:

1. Создание span через Tracer
2. BatchSpanProcessor добавляет span в буфер
3. Проверка лимита буфера и применение Drop Policy
4. Отправка пакета spans через OTLP Exporter

Подсказка: Думайте о логической последовательности: создание → буферизация → проверка → отправка.

Ответ:

- Создание span через Tracer
- BatchSpanProcessor добавляет span в буфер
- Проверка лимита буфера и применение Drop Policy
- Отправка пакета spans через OTLP Exporter

🔧 Шаг 09:

Миссия 9

Расположите действия для настройки управления памятью в OTEL:

1. Задать Max Queue Size
2. Настроить Drop Policy
3. Настроить Batch Timeout
4. Создать BatchSpanProcessor и подключить OTLP Exporter

5. Добавить BatchSpanProcessor к TracerProvider

Подсказка: Сначала настраиваем лимиты и политику, потом подключаем процессор.

Ответ:

- Задать Max Queue Size
- Настроить Drop Policy
- Настроить Batch Timeout
- Создать BatchSpanProcessor и подключить OTLP Exporter
- Добавить BatchSpanProcessor к TracerProvider

Шаг 10:

Миссия 10

Расположите шаги безопасной работы OTEL под нагрузкой:

1. Настроить лимиты очереди и Drop Policy
2. Создать BatchSpanProcessor с OTLP Exporter
3. Добавить BatchSpanProcessor к TracerProvider
4. Создать Tracer и начать создавать spans
5. Проверять буфер и отправку пакетов периодически

Подсказка: Сначала защита памяти, потом процессор, потом использование Tracer.

Ответ:

- Настроить лимиты очереди и Drop Policy
- Создать BatchSpanProcessor с OTLP Exporter
- Добавить BatchSpanProcessor к TracerProvider
- Создать Tracer и начать создавать spans
- Проверять буфер и отправку пакетов периодически

Модуль 4 - Расширенные техники для высоконагруженных приложений

Урок 1 - Асинхронная обработка spans и Batch Exporting

Шаг 01:

Миссия 1

Как BatchSpanProcessor помогает OTEL работать асинхронно?

Подсказка: Подумайте, зачем нужно собирать spans в пакет и отправлять их позже.

Ответ:

- **Буферизует spans и отправляет их пакетно в фоне**
- Отправляет каждый span сразу синхронно
- Сохраняет все spans в памяти без ограничений

Шаг 02:

Миссия 2

Что делает OTLP Exporter при асинхронной обработке spans?

Подсказка: Подумайте о том, как данные передаются из приложения к серверу.

Ответ:

- **Отправляет собранные пакеты spans на сервер без блокировки основного потока**
- Синхронно сохраняет spans в файл
- Создает новые spans вместо отправки старых

Шаг 03:

Миссия 3

Как BatchSpanProcessor управляет частотой отправки spans?

Подсказка: Подумайте о таймере и интервалах экспорта.

Ответ:

- **Отправляет пакеты через заданные интервалы времени или при заполнении буфера**
- Отправляет каждый span мгновенно
- Игнорирует интервал и буферизацию

Шаг 04:

Миссия 4

Что произойдет, если буфер BatchSpanProcessor переполнится?

Подсказка: Подумайте о Drop Policy и управлении памятью.

Ответ:

- **Старые spans могут быть отброшены согласно Drop Policy**
- Приложение аварийно завершится
- Буфер продолжит расти без ограничений

Шаг 05:

Миссия 5

Сопоставьте компоненты OTEL и их роль при асинхронной обработке spans:

Ответ:

- BatchSpanProcessor
 - OTLP Exporter
 - Буфер
 - Drop Policy
1. Отправляет spans на сервер без блокировки основного потока
 2. Собирает spans в пакеты перед отправкой
 3. Хранит spans временно перед отправкой
 4. Стратегия удаления старых spans при переполнении буфера

Подсказка: Подумайте о том, как данные проходят путь от приложения к серверу.

- **BatchSpanProcessor** — Собирает spans в пакеты перед отправкой
- **OTLP Exporter** — Отправляет spans на сервер без блокировки основного потока
- **Буфер** — Хранит spans временно перед отправкой
- **Drop Policy** — Стратегия удаления старых spans при переполнении буфера

Шаг 06:

Миссия 6

Сопоставьте термины с их значением:

Ответ:

- Batch Timeout
 - Max Queue Size
 - Queue Overflow
1. Максимальное время ожидания перед отправкой пакета spans
 2. Лимит на количество spans в очереди
 3. Ситуация, когда буфер превышает лимит

Подсказка: Подумайте, как OTEL контролирует скорость отправки и память.

- **Batch Timeout** — Максимальное время ожидания перед отправкой пакета spans
- **Max Queue Size** — Лимит на количество spans в очереди
- **Queue Overflow** — Ситуация, когда буфер превышает лимит

Шаг 07:

Миссия 7

Сопоставьте последовательность действий при асинхронной обработке span:

Ответ:

- Tracer
 - BatchSpanProcessor
 - OTLP Exporter
1. Создаёт spans для операций приложения
 2. Буферизует и пакетирует spans
 3. Отправляет spans на сервер в фоне

Подсказка: Думайте о цепочке: генерация → буферизация → отправка.

- **Tracer** — Создаёт spans для операций приложения
- **BatchSpanProcessor** — Буферизует и пакетирует spans
- **OTLP Exporter** — Отправляет spans на сервер в фоне

Шаг 08:

Миссия 8

Расположите шаги обработки span при асинхронной работе OTEL:

1. Создание span через Tracer
2. BatchSpanProcessor добавляет span в буфер
3. Проверка лимита буфера и применение Drop Policy
4. Отправка пакета spans через OTLP Exporter

Подсказка: Сначала создаём span, затем буферизация, проверка лимитов, и в конце экспорт.

Ответ:

- Создание span через Tracer
- BatchSpanProcessor добавляет span в буфер
- Проверка лимита буфера и применение Drop Policy
- Отправка пакета spans через OTLP Exporter

🔧 Шаг 09:

Миссия 9

Расположите шаги настройки BatchSpanProcessor и OTLP Exporter:

1. Настроить Max Queue Size и Drop Policy
2. Настроить Batch Timeout
3. Создать BatchSpanProcessor и подключить OTLP Exporter
4. Добавить BatchSpanProcessor к TracerProvider

Подсказка: Сначала настраиваем параметры безопасности и лимитов, потом создаём процессор и подключаем его к TracerProvider.

Ответ:

- Настроить Max Queue Size и Drop Policy
- Настроить Batch Timeout
- Создать BatchSpanProcessor и подключить OTLP Exporter
- Добавить BatchSpanProcessor к TracerProvider

📖 Урок 2 - Ограничение объёма span-ов (Sampling)

🔧 Шаг 01:

Миссия 1

Что делает Sampler в OpenTelemetry?

Подсказка: Подумайте, зачем нужно ограничивать количество span-ов в системе.

Ответ:

- **Решает, какие span-ы создавать и отправлять**
- Создаёт все span-ы без ограничений
- Сохраняет span-ы только локально без экспорта

🔧 Шаг 02:

Миссия 2

Почему Sampling важен в высоконагруженных приложениях?

Подсказка: Подумайте о памяти и сетевых ресурсах.

Ответ:

- **Позволяет уменьшить количество span-ов, экономя память и сеть**
- Увеличивает скорость создания span-ов до максимума
- Удаляет старые span-ы после экспорта

🔧 Шаг 03:

Миссия 3

Как работает вероятность создания span при использовании ProbabilitySampler?

Подсказка: Подумайте о случайном выборе и шансах.

Ответ:

- **Каждый span создаётся с определённой вероятностью**
- Все span-ы создаются, но часть потом отбрасывается
- Span создаётся только при высоких нагрузках

Шаг 04:

Миссия 4

Что происходит, если использовать AlwaysOnSampler?

Подсказка: Подумайте о количестве span-ов и нагрузке.

Ответ:

- **Все span-ы создаются без исключений**
- Ни один span не создаётся
- Span-ы создаются только с определённой вероятностью

Шаг 05:

Миссия 5

Сопоставьте тип Sampler с его поведением:

Ответ:

- AlwaysOnSampler
 - AlwaysOffSampler
 - ProbabilitySampler
 - TraceldRatioBasedSampler
1. Создаёт все span-ы
 2. Никогда не создаёт span-ы
 3. Создаёт span с заданной вероятностью
 4. Использует Traceld для определения, сохранять span или нет

Подсказка: Подумайте о стратегиях: всегда, никогда, случайно и по идентификатору.

- **AlwaysOnSampler** — Создаёт все span-ы
- **AlwaysOffSampler** — Никогда не создаёт span-ы
- **ProbabilitySampler** — Создаёт span с заданной вероятностью
- **TraceldRatioBasedSampler** — Использует Traceld для определения, сохранять span или нет

Шаг 06:

Миссия 6

Сопоставьте термин с примером использования:

Ответ:

- Sampling
 - Span
 - Trace
 - ProbabilitySampler
1. Ограничение количества span-ов для экономии ресурсов
 2. Единичная операция или событие в приложении
 3. Последовательность span-ов, представляющая выполнение запроса
 4. Выбор span-ов с определённой вероятностью

Подсказка: Подумайте о терминах и примерах из реального приложения.

- **Sampling** — Ограничение количества span-ов для экономии ресурсов
- **Span** — Единичная операция или событие в приложении

- **Trace** — Последовательность span-ов, представляющая выполнение запроса
- **ProbabilitySampler** — Выбор span-ов с определённой вероятностью

Шаг 07:

Миссия 7

Сопоставьте стратегию Sampling с её преимуществом:

Ответ:

- AlwaysOnSampler
- AlwaysOffSampler
- ProbabilitySampler
- TracedRatioBasedSampler

1. Позволяет отлаживать полностью без пропусков span-ов
2. Идеально для минимизации нагрузки, ничего не отправляя
3. Снижает нагрузку случайным образом, сохраняя репрезентативность
4. Сохраняет предсказуемость выборки через Traced

Подсказка: Думайте о плюсах каждого подхода в разных сценариях.

- **AlwaysOnSampler** — Позволяет отлаживать полностью без пропусков span-ов
- **AlwaysOffSampler** — Идеально для минимизации нагрузки, ничего не отправляя
- **ProbabilitySampler** — Снижает нагрузку случайным образом, сохраняя репрезентативность
- **TracedRatioBasedSampler** — Сохраняет предсказуемость выборки через Traced

Шаг 08:

Миссия 8

Расположите шаги настройки ProbabilitySampler для Python приложения:

1. Импортировать нужный Sampler из opentelemetry.sdk.trace
2. Задать желаемую вероятность выборки span-ов
3. Создать TracerProvider с выбранным Sampler
4. Добавить TracerProvider к приложению

Подсказка: Сначала импорт, потом настройка параметров, создание провайдера и интеграция.

Ответ:

- Импортировать нужный Sampler из opentelemetry.sdk.trace
- Задать желаемую вероятность выборки span-ов
- Создать TracerProvider с выбранным Sampler
- Добавить TracerProvider к приложению

Шаг 09:

Миссия 9

Расположите шаги от генерации span до применения Sampling:

1. Происходит событие или операция в приложении
2. Tracer создаёт span
3. Sampler решает, сохранять span или нет
4. Span либо отправляется через процессор и экспортер, либо отбрасывается

Подсказка: Думайте о логике: событие → span → решение Sampler → отправка/отброс.

Ответ:

- Происходит событие или операция в приложении
- Tracer создаёт span
- Sampler решает, сохранять span или нет
- Span либо отправляется через процессор и экспортер, либо отбрасывается

Шаг 10:

Миссия 10

Расположите шаги безопасного использования Sampling в высоконагруженном приложении:

1. Определить нагрузку и требования к мониторингу
2. Выбрать тип Sampler и настроить параметры
3. Интегрировать Sampler с TracerProvider
4. Проверить корректность выборки span-ов и нагрузку на систему
5. Запустить приложение с мониторингом и при необходимости скорректировать Sampling

Подсказка: Сначала анализ, потом настройка, интеграция и тестирование.

Ответ:

- Определить нагрузку и требования к мониторингу
- Выбрать тип Sampler и настроить параметры
- Интегрировать Sampler с TracerProvider
- Проверить корректность выборки span-ов и нагрузку на систему
- Запустить приложение с мониторингом и при необходимости скорректировать Sampling

Урок 3 - Атрибуты и события, полезные для профилирования

Шаг 01:

Миссия 1

Что такое атрибуты span-ов в OpenTelemetry?

Подсказка: Атрибуты описывают дополнительные данные о span-е.

Ответ:

- **Ключ-значение, описывающее span**
- Тип span-а (sync/async)
- Время выполнения span-а

Шаг 02:

Миссия 2

Для чего используются события span-ов?

Подсказка: Подумайте о том, что span может содержать динамические точки во времени.

Ответ:

- **Фиксируют моменты времени внутри span-а с дополнительной информацией**
- Определяют имя span-а
- Задают максимальное время жизни span-а

Шаг 03:

Миссия 3

Какие данные обычно хранятся в span-атрибутах для профилирования?

Подсказка: Подумайте о метриках и контексте операции.

Ответ:

- **Имя метода, длительность, размер запроса**
- Версия Python
- IP адрес пользователя
- Количество span-ов в Trace

Шаг 04:

Миссия 4

Почему полезно добавлять события span-ов при профилировании?

Подсказка: События помогают отслеживать ключевые точки во времени.

Ответ:

- **Позволяет видеть, когда происходят важные события внутри span-а**
- Ускоряет выполнение span-а
- Автоматически удаляет span после события

Шаг 05:

Миссия 5

Сопоставьте тип данных с примером, который может храниться в атрибутах span:

Ответ:

- Строка
 - Число
 - Булево
 - Массив
1. Имя функции
 2. Время выполнения в миллисекундах
 3. Флаг ошибки
 4. Список использованных библиотек

Подсказка: Думайте о типе информации, которую удобно хранить в каждом формате.

- **Строка** — Имя функции
- **Число** — Время выполнения в миллисекундах
- **Булево** — Флаг ошибки
- **Массив** — Список использованных библиотек

Шаг 06:

Миссия 6

Сопоставьте событие span с его возможным применением:

Ответ:

- message_sent
- db_query_start

- `cache_hit`
- `exception_occurred`

1. Отслеживание момента отправки сообщения
2. Начало выполнения запроса к базе данных
3. Успешное получение данных из кэша
4. Фиксация исключения

Подсказка: Думайте о том, какие события помогают профилировать выполнение приложения.

- **`message_sent`** — Отслеживание момента отправки сообщения
- **`db_query_start`** — Начало выполнения запроса к базе данных
- **`cache_hit`** — Успешное получение данных из кэша
- **`exception_occurred`** — Фиксация исключения

Шаг 07:

Миссия 7

Сопоставьте span-атрибут с информацией, полезной для профилирования:

Ответ:

- `http.method`
 - `db.rows_returned`
 - `cpu_usage`
 - `error_flag`
1. Тип HTTP запроса (GET, POST)
 2. Количество возвращённых строк из базы
 3. Использование CPU во время span
 4. Наличие ошибки во время span

Подсказка: Подумайте, какая метрика соответствует какому атрибуту.

- **`http.method`** — Тип HTTP запроса (GET, POST)
- **`db.rows_returned`** — Количество возвращённых строк из базы
- **`cpu_usage`** — Использование CPU во время span
- **`error_flag`** — Наличие ошибки во время span

Шаг 08:

Миссия 8

Расположите шаги добавления событий и атрибутов для профилирования span:

1. Создать span для операции
2. Добавить нужные атрибуты
3. Добавить события с временными метками
4. Закончить span и отправить его экспортеру

Подсказка: Сначала span создаётся, потом дополняется, потом завершается.

Ответ:

- Создать span для операции
- Добавить нужные атрибуты
- Добавить события с временными метками
- Закончить span и отправить его экспортеру

Урок 4 - Интеграция с базами данных и сторонними сервисами

Шаг 01:

Миссия 1

Зачем интегрировать OpenTelemetry со сторонними базами данных?

Подсказка: Подумайте, какие данные важно отслеживать при работе с базой.

Ответ:

- **Чтобы отслеживать производительность и ошибки запросов**
- Чтобы ускорить выполнение SQL-запросов
- Чтобы автоматически создавать индексы

Шаг 02:

Миссия 2

Что лучше всего логировать при вызове стороннего API через OpenTelemetry?

Подсказка: Подумайте о параметрах запроса и ответе сервиса.

Ответ:

- **Метод API, статус ответа, время выполнения**
- Только тело ответа
- IP адрес пользователя

Шаг 03:

Миссия 3

Почему важно добавлять атрибуты span-ов при интеграции с базой данных?

Подсказка: Атрибуты помогают понять, что именно происходит внутри span-а.

Ответ:

- **Чтобы фиксировать SQL-запросы, таблицы и количество строк**
- Чтобы ускорить выполнение базы данных
- Чтобы автоматически создавать резервные копии

Шаг 04:

Миссия 4

Как события span-ов помогают при мониторинге сторонних сервисов?

Подсказка: Подумайте о моментах начала и конца вызова API.

Ответ:

- **Фиксируют ключевые моменты, такие как отправка запроса и получение ответа**
- Автоматически исправляют ошибки API
- Сокращают время отклика сервиса

Шаг 05:

Миссия 5

Сопоставьте span-атрибут с примером информации для базы данных:

Ответ:

- db.system

- db.statement
- db.user
- db.rows_returned

1. Тип СУБД (PostgreSQL, MySQL)
2. SQL-запрос, который выполняется
3. Имя пользователя базы данных
4. Количество строк, возвращённых запросом

Подсказка: Подумайте, какой атрибут отвечает за какую информацию.

- **db.system** — Тип СУБД (PostgreSQL, MySQL)
- **db.statement** — SQL-запрос, который выполняется
- **db.user** — Имя пользователя базы данных
- **db.rows_returned** — Количество строк, возвращённых запросом

Шаг 06:

Миссия 6

Сопоставьте события span с ситуациями при работе со сторонними сервисами:

Ответ:

- request_sent
 - response_received
 - exception_occurred
 - cache_hit
1. Отправка запроса к API
 2. Получение ответа от API
 3. Произошла ошибка при вызове сервиса
 4. Данные успешно получены из кэша

Подсказка: События фиксируют важные точки во времени.

- **request_sent** — Отправка запроса к API
- **response_received** — Получение ответа от API
- **exception_occurred** — Произошла ошибка при вызове сервиса
- **cache_hit** — Данные успешно получены из кэша

Шаг 07:

Миссия 7

Сопоставьте интеграцию OpenTelemetry с библиотекой с её эффектом:

Ответ:

- psycopg2
 - requests
 - redis-py
 - aiohttp
1. Отслеживание SQL-запросов PostgreSQL
 2. Отслеживание HTTP-запросов к внешним API
 3. Отслеживание операций с Redis
 4. Асинхронные HTTP-запросы

Подсказка: Каждая библиотека имеет своё назначение.

- **psycopg2** — Отслеживание SQL-запросов PostgreSQL
- **requests** — Отслеживание HTTP-запросов к внешним API
- **redis-py** — Отслеживание операций с Redis
- **aiohttp** — Асинхронные HTTP-запросы

Шаг 08:

Миссия 8

Расположите шаги интеграции OpenTelemetry с базой данных:

1. Установить OTEL-инструментирование для используемой библиотеки
2. Создать и настроить Tracer
3. Добавлять span-ы вокруг операций с базой
4. Экспортировать собранные span-ы в бэкенд для анализа

Подсказка: Сначала настраивается библиотека, потом Tracer, затем span-ы и экспорт.

Ответ:

- Установить OTEL-инструментирование для используемой библиотеки
- Создать и настроить Tracer
- Добавлять span-ы вокруг операций с базой
- Экспортировать собранные span-ы в бэкенд для анализа

Шаг 09:

Миссия 9

Расположите шаги профилирования стороннего API с помощью OpenTelemetry:

1. Создать span для API-вызова
2. Добавить события: request_sent, response_received
3. Добавить атрибуты: метод, URL, статус
4. Закончить span и экспортировать данные

Подсказка: Сначала создаём span, затем добавляем события и атрибуты, в конце завершаем и экспортируем.

Ответ:

- Создать span для API-вызова
- Добавить события: request_sent, response_received
- Добавить атрибуты: метод, URL, статус
- Закончить span и экспортировать данные

Шаг 10:

Миссия 10

Расположите шаги анализа данных из баз и API для оптимизации приложения:

1. Собрать span-ы с атрибутами и событиями
2. Фильтровать по важным метрикам
3. Визуализировать длительность и ошибки запросов
4. Использовать вывод для оптимизации кода и запросов

Подсказка: Анализ начинается с данных, потом фильтрация, визуализация и, наконец, оптимизация.

Ответ:

- Собрать span-ы с атрибутами и событиями
- Фильтровать по важным метрикам
- Визуализировать длительность и ошибки запросов
- Использовать вывод для оптимизации кода и запросов

Урок 5 - Error tracking и паники (Recovery + span.SetStatus)

Шаг 01:

Миссия 1

Что делает span.SetStatus при отслеживании ошибок в OpenTelemetry?

Подсказка: Подумайте, как пометить span как успешный или с ошибкой.

Ответ:

- **Устанавливает статус span-а как OK или ERROR**
- Автоматически исправляет ошибки в коде
- Создаёт новый span при каждой ошибке

Шаг 02:

Миссия 2

Почему важно использовать Recovery при панике в Python?

Подсказка: Подумайте, что происходит с приложением, если ошибка не обработана.

Ответ:

- **Чтобы приложение продолжало работу и ошибки фиксировались в span**
- Чтобы ускорить выполнение программы
- Чтобы автоматически исправлять ошибки в коде

Шаг 03:

Миссия 3

Что лучше всего логировать при ошибках в внешнем API с OpenTelemetry?

Подсказка: Подумайте о том, какие данные помогут понять причину ошибки.

Ответ:

- **Статус ответа, тело ошибки, время вызова**
- Только тело запроса
- Только URL сервиса

Шаг 04:

Миссия 4

Зачем фиксировать exception_occurred как событие span-а?

Подсказка: Подумайте, что важно видеть при мониторинге ошибок.

Ответ:

- **Чтобы видеть момент возникновения ошибки и её детали**
- Чтобы исправлять ошибки автоматически

- Чтобы завершить работу приложения быстрее

Шаг 05:

Миссия 5

Сопоставьте статус span с его описанием:

Ответ:

- OK
 - ERROR
 - UNSET
1. Всё прошло успешно
 2. Произошла ошибка в span-е
 3. Статус не установлен

Подсказка: Статус span показывает результат операции.

- **OK** — Всё прошло успешно
- **ERROR** — Произошла ошибка в span-е
- **UNSET** — Статус не установлен

Шаг 06:

Миссия 6

Сопоставьте событие span с его применением:

Ответ:

- exception_occurred
 - panic_recovered
 - request_sent
 - response_received
1. Зафиксирована ошибка в коде
 2. Паника была обработана и программа продолжила работу
 3. Запрос отправлен во внешнюю систему
 4. Получен ответ от внешней системы

Подсказка: События помогают понять, что произошло в процессе выполнения span-а.

- **exception_occurred** — Зафиксирована ошибка в коде
- **panic_recovered** — Паника была обработана и программа продолжила работу
- **request_sent** — Запрос отправлен во внешнюю систему
- **response_received** — Получен ответ от внешней системы

Шаг 07:

Миссия 7

Сопоставьте тип ошибки с подходящей реакцией span:

Ответ:

- Сетевая ошибка
- Исключение в коде
- Ошибочный HTTP-статус

- Паника приложения

1. Установить статус ERROR и записать exception_occurred
2. Установить статус ERROR и логировать детали HTTP
3. Использовать Recovery и пометить span как ERROR
4. Использовать span.SetStatus и логировать трассировку ошибки

Подсказка: Разные типы ошибок требуют разных действий span-а.

- **Сетевая ошибка** — Установить статус ERROR и записать exception_occurred
- **Исключение в коде** — Использовать span.SetStatus и логировать трассировку ошибки
- **Ошибочный HTTP-статус** — Установить статус ERROR и логировать детали HTTP
- **Паника приложения** — Использовать Recovery и пометить span как ERROR

Шаг 08:

Миссия 8

Расположите шаги обработки ошибки в span в правильном порядке:

1. Произошла ошибка или паника
2. Использовать Recovery (если паника)
3. Установить span.SetStatus(ERROR)
4. Добавить событие exception_occurred
5. Продолжить выполнение программы

Подсказка: Сначала фиксируем ошибку, затем защищаем приложение, помечаем статус и логируем событие.

Ответ:

- Произошла ошибка или паника
- Использовать Recovery (если паника)
- Установить span.SetStatus(ERROR)
- Добавить событие exception_occurred
- Продолжить выполнение программы

Шаг 09:

Миссия 9

Расположите шаги логирования ошибок при внешнем API:

1. Создать span вокруг вызова API
2. Добавить события request_sent и response_received
3. Если ошибка, установить span.SetStatus(ERROR) и exception_occurred
4. Закрыть span и экспортировать данные

Подсказка: Сначала создаём span, фиксируем события, помечаем ошибки, затем закрываем span.

Ответ:

- Создать span вокруг вызова API
- Добавить события request_sent и response_received
- Если ошибка, установить span.SetStatus(ERROR) и exception_occurred
- Закрыть span и экспортировать данные

Модуль 5 - Инструменты визуализации и анализа

Урок 1 - Цели научиться эффективно работать с данными OTEL.

Шаг 01:

Миссия 1

Для чего используют дашборды OpenTelemetry в высоконагруженных приложениях?
Подсказка: Подумайте, как быстро понять состояние системы и найти узкие места.

Ответ:

- Для визуализации метрик и состояния приложений
- Для автоматического исправления кода
- Для ускорения работы базы данных

Шаг 02:

Миссия 2

Что важно включать при построении графиков производительности в OTEL?
Подсказка: Подумайте, какие данные помогут понять нагрузку на сервис.

Ответ:

- Метрики **latency, throughput, error rate**
- Только текст логов
- Случайные значения переменных

Шаг 03:

Миссия 3

Зачем добавлять теги и атрибуты к span-ам в OTEL?
Подсказка: Подумайте, как найти проблему быстрее, когда их много.

Ответ:

- Чтобы фильтровать и анализировать события по ключевым параметрам
- Чтобы уменьшить количество span-ов
- Чтобы автоматически исправлять ошибки

Шаг 04:

Миссия 4

Что лучше всего использовать для анализа ошибок и узких мест в OTEL?
Подсказка: Подумайте, какой инструмент объединяет трассировки и метрики.

Ответ:

- Трассировки с метками и событиями **span**
- Только лог-файлы
- Только диаграммы нагрузки CPU

Шаг 05:

Миссия 5

Сопоставьте тип данных OTEL с их применением:

Ответ:

- Метрики
- Трассировки
- Логи

1. Измерение производительности и загрузки сервиса
2. Отслеживание последовательности вызовов и ошибок
3. Детали событий и состояния объектов

Подсказка: Подумайте, что измеряет, что отслеживает, а что записывает детали.

- **Метрики** — Измерение производительности и загрузки сервиса
- **Трассировки** — Отслеживание последовательности вызовов и ошибок
- **Логи** — Детали событий и состояния объектов

 Шаг 06:

Миссия 6

Сопоставьте визуальные элементы OTEL с их назначением:

Ответ:

- Графики времени ответа
- Тепловые карты нагрузки
- Диаграммы трассировок

1. Показывают задержки и латентность запросов
2. Отображают узкие места и горячие точки
3. Визуализируют последовательность операций и ошибки

Подсказка: Разные графики нужны для разных аспектов анализа.

- **Графики времени ответа** — Показывают задержки и латентность запросов
- **Тепловые карты нагрузки** — Отображают узкие места и горячие точки
- **Диаграммы трассировок** — Визуализируют последовательность операций и ошибки

 Шаг 07:

Миссия 7

Сопоставьте цели анализа OTEL с действиями разработчика:

Ответ:

- Поиск узких мест
- Отслеживание ошибок
- Оптимизация ресурсов

1. Смотрим графики latency и throughput
2. Используем события span и логи для диагностики
3. Анализируем использование CPU, памяти и других метрик

Подсказка: Каждая цель требует своего типа анализа.

- **Поиск узких мест** — Смотрим графики latency и throughput
- **Отслеживание ошибок** — Используем события span и логи для диагностики
- **Оптимизация ресурсов** — Анализируем использование CPU, памяти и других метрик

Шаг 08:

Миссия 8

Расположите шаги построения дашборда для анализа OTEL данных:

1. Определяем цели мониторинга
2. Выбираем метрики, трассировки и логи
3. Строим визуализации (графики, диаграммы, карты)
4. Настраиваем фильтры и алерты
5. Анализируем данные и принимаем решения

Подсказка: Сначала цель, затем данные, визуализация, фильтры и анализ.

Ответ:

- Определяем цели мониторинга
- Выбираем метрики, трассировки и логи
- Строим визуализации (графики, диаграммы, карты)
- Настраиваем фильтры и алерты
- Анализируем данные и принимаем решения

Шаг 09:

Миссия 9

Расположите шаги анализа ошибок с OTEL:

1. Собираем данные span, метрик и логов
2. Фильтруем по типу ошибки и сервису
3. Строим визуализации для понимания паттернов
4. Выявляем причины и точки оптимизации
5. Предлагаем решения и оптимизируем код

Подсказка: От сбора данных до действий — последовательный процесс.

Ответ:

- Собираем данные span, метрик и логов
- Фильтруем по типу ошибки и сервису
- Строим визуализации для понимания паттернов
- Выявляем причины и точки оптимизации
- Предлагаем решения и оптимизируем код

Шаг 10:

Миссия 10

Расположите шаги работы с OTEL данными для оптимизации производительности:

1. Определяем ключевые показатели (latency, throughput, errors)
2. Собираем span-ы и метрики из сервисов
3. Визуализируем данные на графиках и дашбордах
4. Идентифицируем узкие места и аномалии
5. Вносим изменения и повторяем процесс мониторинга

Подсказка: Анализ всегда начинается с выбора показателей, затем сбор, визуализация, выявление проблем и корректировка.

Ответ:

- Определяем ключевые показатели (latency, throughput, errors)
- Собираем span-ы и метрики из сервисов
- Визуализируем данные на графиках и дашбордах
- Идентифицируем узкие места и аномалии
- Вносим изменения и повторяем процесс мониторинга

Урок 2 - Jaeger Tempo Grafana OpenTelemetry Collector

Шаг 01:

Миссия 1

Для чего обычно используют Jaeger в высоконагруженных приложениях?

Подсказка: Подумайте о том, как отследить цепочку запросов между сервисами.

Ответ:

- **Для распределённого трассирования запросов**
- Для хранения логов
- Для мониторинга CPU и памяти

Шаг 02:

Миссия 2

Что такое Tempo в контексте OpenTelemetry?

Подсказка: Подумайте о системе, которая собирает трассировки для анализа.

Ответ:

- **Система для хранения и поиска распределённых трассировок**
- Графическая панель для метрик
- Инструмент автоматической оптимизации кода

Шаг 03:

Миссия 3

Как Grafana помогает работать с OpenTelemetry данными?

Подсказка: Подумайте о визуальном отображении метрик и событий.

Ответ:

- **Позволяет строить дашборды и графики по метрикам и трассировкам**
- Сохраняет данные на диск
- Исправляет ошибки в приложении автоматически

Шаг 04:

Миссия 4

Зачем нужен OpenTelemetry Collector?

Подсказка: Подумайте о централизованном сборе данных из разных сервисов.

Ответ:

- **Для сбора, обработки и экспорта метрик, логов и трассировок**
- Для генерации случайных метрик
- Для оптимизации памяти приложения

Шаг 05:

Миссия 5

Сопоставьте инструмент с его основной функцией:

Ответ:

- Jaeger
- Tempo
- Grafana
- OTEL Collector

1. Визуализация метрик и графиков
2. Хранение и поиск распределённых трассировок
3. Сбор, обработка и экспорт телеметрии
4. Распределённое трассирование запросов

Подсказка: Каждый инструмент имеет свою уникальную роль в цепочке наблюдаемости.

- **Jaeger** — Распределённое трассирование запросов
- **Tempo** — Хранение и поиск распределённых трассировок
- **Grafana** — Визуализация метрик и графиков
- **OTEL Collector** — Сбор, обработка и экспорт телеметрии

Шаг 06:

Миссия 6

Сопоставьте задачи с инструментом OTEL:

Ответ:

- Хранение трассировок для долгосрочного анализа
- Сбор телеметрии из нескольких сервисов
- Визуализация графиков задержек
- Анализ последовательности вызовов API

1. Tempo
2. OTEL Collector
3. Grafana
4. Jaeger

Подсказка: Подумайте, где хранится, что собирается, что визуализируется и что анализируется.

- **Хранение трассировок для долгосрочного анализа** — Tempo
- **Сбор телеметрии из нескольких сервисов** — OTEL Collector
- **Визуализация графиков задержек** — Grafana
- **Анализ последовательности вызовов API** — Jaeger

Шаг 07:

Миссия 7

Сопоставьте формат данных с правильным примером:

Ответ:

- Метрики
- Трассировки

- Логи

1. CPU usage 70%, memory 2GB
2. Span start, span end, child spans
3. Error: database connection failed

Подсказка: Метрики — численные показатели, трассировки — последовательность операций, логи — текстовые события.

- **Метрики** — CPU usage 70%, memory 2GB
- **Трассировки** — Span start, span end, child spans
- **Логи** — Error: database connection failed

Шаг 08:

Миссия 8

Расположите шаги использования OTEL Collector для передачи данных в Jaeger:

1. Настраиваем Collector для приёма телеметрии
2. Обработываем и фильтруем span-ы
3. Экспортируем данные в Jaeger
4. Проверяем трассировки в Jaeger UI

Подсказка: Сначала приём данных, затем обработка, экспорт и проверка.

Ответ:

- Настраиваем Collector для приёма телеметрии
- Обработываем и фильтруем span-ы
- Экспортируем данные в Jaeger
- Проверяем трассировки в Jaeger UI

Шаг 09:

Миссия 9

Расположите шаги анализа данных в Grafana:

1. Подключаем источник данных (OTEL Collector / Tempo / Jaeger)
2. Создаём дашборд
3. Выбираем метрики или трассировки
4. Строим графики и панели
5. Интерпретируем данные и принимаем решения

Подсказка: От источника к визуализации и анализу.

Ответ:

- Подключаем источник данных (OTEL Collector / Tempo / Jaeger)
- Создаём дашборд
- Выбираем метрики или трассировки
- Строим графики и панели
- Интерпретируем данные и принимаем решения

Шаг 10:

Миссия 10

Расположите шаги от создания трассировки до её анализа в Tempo и Grafana:

1. Создаём span в приложении
2. Отправляем span в OTEL Collector
3. Собираем и храним трассировки в Tempo
4. Подключаем Tempo к Grafana
5. Анализируем трассировки и выявляем узкие места

Подсказка: Логика: генерация → сбор → хранение → визуализация → анализ.

Ответ:

- Создаём span в приложении
- Отправляем span в OTEL Collector
- Собираем и храним трассировки в Tempo
- Подключаем Tempo к Grafana
- Анализируем трассировки и выявляем узкие места

Урок 3 - Просмотр трасс и зависимостей микросервисов

Шаг 01:

Миссия 1

Что позволяет понять просмотр трасс в микросервисной архитектуре?

Подсказка: Думайте о том, как один запрос проходит через разные сервисы.

Ответ:

- **Последовательность вызовов между сервисами**
- Только количество ошибок в приложении
- Время отклика одной функции без контекста

Шаг 02:

Миссия 2

Какая информация чаще всего содержится в span при трассировании?

Подсказка: Подумайте о метках времени и связях между сервисами.

Ответ:

- **Начало и конец операции, теги и родительский span**
- Содержимое базы данных
- Системные логи операционной системы

Шаг 03:

Миссия 3

Как визуализация зависимостей микросервисов помогает разработчику?

Подсказка: Представьте карту связей между сервисами.

Ответ:

- **Быстро находить узкие места и зависимости между сервисами**
- Сохранять кодовые коммиты
- Ускорять компиляцию приложения

🔮 Шаг 04:

Миссия 4

Что отображает граф зависимостей микросервисов?

Подсказка: Думайте о стрелках, соединяющих разные сервисы.

Ответ:

- **Взаимосвязи и потоки запросов между сервисами**
- Внутренние структуры классов Python
- Распределение памяти на сервере

🔮 Шаг 05:

Миссия 5

Сопоставьте элементы трассировки с их назначением:

Ответ:

- Span
 - Trace
 - Tag
 - Parent Span
1. Отдельная операция или шаг внутри запроса
 2. Все связанные span-ы одного запроса
 3. Ключ-значение с дополнительной информацией
 4. Связь с предыдущей операцией

Подсказка: Подумайте о структуре трассировки как о дереве.

- **Span** — Отдельная операция или шаг внутри запроса
- **Trace** — Все связанные span-ы одного запроса
- **Tag** — Ключ-значение с дополнительной информацией
- **Parent Span** — Связь с предыдущей операцией

🔮 Шаг 06:

Миссия 6

Сопоставьте визуальные элементы с их функцией:

Ответ:

- Узел на графе
 - Линия между узлами
 - Цвет узла
 - Толщина линии
1. Отдельный сервис
 2. Взаимодействие между сервисами
 3. Статус сервиса (ошибка, нормальное состояние)
 4. Объём трафика или задержка

Подсказка: Подумайте, как визуальные подсказки помогают быстро понять систему.

- **Узел на графе** — Отдельный сервис
- **Линия между узлами** — Взаимодействие между сервисами
- **Цвет узла** — Статус сервиса (ошибка, нормальное состояние)

- **Толщина линии** — Объем трафика или задержка

Шаг 07:

Миссия 7

Сопоставьте термин с примером в микросервисной трассировке:

Ответ:

- Client Request
 - HTTP Call
 - Database Query
 - Service Response
1. Пользователь запрашивает страницу
 2. Сервис А вызывает сервис В по HTTP
 3. `SELECT * FROM users`
 4. Сервис возвращает результат пользователю

Подсказка: Подумайте о полном цикле запроса через систему.

- **Client Request** — Пользователь запрашивает страницу
- **HTTP Call** — Сервис А вызывает сервис В по HTTP
- **Database Query** — `SELECT * FROM users`
- **Service Response** — Сервис возвращает результат пользователю

Шаг 08:

Миссия 8

Расположите шаги анализа зависимостей микросервисов в правильном порядке:

1. Собираем трассировки из всех сервисов
2. Строим граф зависимостей
3. Выделяем узкие места и медленные сервисы
4. Принимаем решения по оптимизации

Подсказка: Логика: сбор → визуализация → анализ → действия.

Ответ:

- Собираем трассировки из всех сервисов
- Строим граф зависимостей
- Выделяем узкие места и медленные сервисы
- Принимаем решения по оптимизации

Шаг 09:

Миссия 9

Расположите этапы поиска проблемы в трассировке:

1. Открываем трассировку с медленным запросом
2. Смотрим последовательность span-ов
3. Определяем, какой сервис задерживает ответ
4. Проверяем логи и метрики сервиса
5. Вносим изменения и повторяем трассировку

Подсказка: Сначала наблюдаем, потом анализируем, потом действуем.

Ответ:

- Открываем трассировку с медленным запросом
- Смотрим последовательность span-ов
- Определяем, какой сервис задерживает ответ
- Проверяем логи и метрики сервиса
- Вносим изменения и повторяем трассировку

Урок 4 - Метрики vs spans что и когда использовать

Шаг 01:

Миссия 1

Что лучше использовать для отслеживания количества запросов в сервисе?

Подсказка: Подумайте, что считается агрегированным числом за время.

Ответ:

- **Метрики (Counters)**
- Span для каждого запроса
- Логи приложения

Шаг 02:

Миссия 2

Что лучше использовать для измерения задержки конкретного запроса?

Подсказка: Думайте о временных метках начала и конца операции.

Ответ:

- **Span**
- Гистограмма метрик
- Текстовый лог с временем

Шаг 03:

Миссия 3

Для чего чаще всего используют гистограммы в метриках?

Подсказка: Подумайте о распределении значений, а не их количестве.

Ответ:

- **Для анализа распределения времени отклика**
- Для хранения логов ошибок
- Для отображения структуры кода

Шаг 04:

Миссия 4

Что помогает понять путь запроса через несколько сервисов?

Подсказка: Думайте о «следе», который оставляет запрос.

Ответ:

- **Span**
- Counter метрики
- Gauge метрики

Шаг 05:

Миссия 5

Сопоставьте термин с правильным использованием:

Ответ:

- Counter
- Gauge
- Histogram
- Span

1. Отслеживание общего количества событий
2. Фиксирование текущего значения ресурса
3. Анализ распределения задержек
4. Проследить путь конкретного запроса

Подсказка: Подумайте, что хранит агрегированное значение, а что индивидуальную запись.

- **Counter** — Отслеживание общего количества событий
- **Gauge** — Фиксирование текущего значения ресурса
- **Histogram** — Анализ распределения задержек
- **Span** — Проследить путь конкретного запроса

Шаг 06:

Миссия 6

Сопоставьте тип метрик с примером использования:

Ответ:

- Counter
- Gauge
- Histogram

1. Количество успешных логинов пользователей
2. Текущий объём используемой памяти
3. Распределение времени отклика API

Подсказка: Подумайте, какие метрики измеряют «текущее» состояние, а какие — накопленные события.

- **Counter** — Количество успешных логинов пользователей
- **Gauge** — Текущий объём используемой памяти
- **Histogram** — Распределение времени отклика API

Шаг 07:

Миссия 8

Расположите этапы выбора между метрикой и span для мониторинга:

1. Определяем, хотим ли мы агрегированное значение или отдельный путь запроса
2. **Если нужен путь запроса через сервисы** — выбираем span
3. **Если нужно агрегированное число или распределение** — выбираем метрику
4. Настраиваем сбор данных и визуализацию

Подсказка: Сначала думаем о цели, потом о типе данных, потом настраиваем сбор.

Ответ:

- Определяем, хотим ли мы агрегированное значение или отдельный путь запроса
- **Если нужен путь запроса через сервисы** — выбираем span
- **Если нужно агрегированное число или распределение** — выбираем метрику
- Настраиваем сбор данных и визуализацию

 Шаг 08:

Миссия 8

Расположите шаги анализа производительности API с помощью метрик и spans:

1. Собираем spans для отдельных запросов
2. Собираем метрики времени отклика и ошибок
3. Анализируем задержки с помощью spans
4. Выявляем узкие места по метрикам
5. Принимаем решения по оптимизации

Подсказка: Сначала собираем данные, потом анализируем, потом действуем.

Ответ:

- Собираем spans для отдельных запросов
- Собираем метрики времени отклика и ошибок
- Анализируем задержки с помощью spans
- Выявляем узкие места по метрикам
- Принимаем решения по оптимизации

 Шаг 09:

Миссия 9

Расположите этапы принятия решения, когда использовать метрики или spans:

1. Определяем цель мониторинга (агрегированное число или детальный путь запроса)
2. Выбираем тип данных: метрика или span
3. Настраиваем сбор данных и визуализацию
4. Смотрим результаты и корректируем настройку при необходимости

Подсказка: Логика: цель → выбор инструмента → настройка → проверка.

Ответ:

- Определяем цель мониторинга (агрегированное число или детальный путь запроса)
- Выбираем тип данных: метрика или span
- Настраиваем сбор данных и визуализацию
- Смотрим результаты и корректируем настройку при необходимости

 Урок 5 - Анализ узких мест и горячих точек

 Шаг 01:

Миссия 1

Что первым делом стоит изучить при выявлении узкого места в сервисе?

Подсказка: Думайте о самом медленном участке в цепочке выполнения запроса.

Ответ:

- **Span с самой большой задержкой**

- Общее количество запросов
- Размер лог-файла

🔮 Шаг 02:

Миссия 2

Какой метрикой удобно определять горячие точки по нагрузке на сервис?

Подсказка: Смотрите на показатели, которые увеличиваются при росте нагрузки.

Ответ:

- **Gauge текущей нагрузки или CPU**
- Counter количества ошибок за день
- Histogram времени отклика отдельного запроса

🔮 Шаг 03:

Миссия 3

Что удобнее использовать для анализа задержек на разных этапах запроса?

Подсказка: Думайте о подробной временной трассировке каждого запроса.

Ответ:

- **Span**
- Counter
- Gauge

🔮 Шаг 04:

Миссия 4

Что поможет визуально определить горячие точки между микросервисами?

Подсказка: Ищите визуализацию, где видно поток запроса и задержки.

Ответ:

- **Сервисная карта с цветовой индикацией задержек**
- Таблица логов ошибок
- Счётчик количества вызовов API

🔮 Шаг 05:

Миссия 5

Сопоставьте инструмент с его использованием для анализа горячих точек:

Ответ:

- Span
 - Histogram метрики времени отклика
 - Gauge загрузки CPU
 - Сервисная карта
1. Видим путь запроса и задержки на каждом этапе
 2. Анализируем распределение времени отклика
 3. Следим за загрузкой ресурсов в реальном времени
 4. Визуально выявляем узкие места между сервисами

Подсказка: Подумайте, что детализирует отдельный запрос, а что — агрегированные данные.

- **Span** — Видим путь запроса и задержки на каждом этапе
- **Histogram метрики времени отклика** — Анализируем распределение времени отклика
- **Gauge загрузки CPU** — Следим за загрузкой ресурсов в реальном времени
- **Сервисная карта** — Визуально выявляем узкие места между сервисами

🔮 Шаг 06:

Миссия 6

Сопоставьте проблему с показателем для её выявления:

Ответ:

- Долгий отклик запроса
 - Перегруженный сервис
 - Высокий процент ошибок
1. Span с большой задержкой
 2. Gauge или CPU/Memory метрика
 3. Counter ошибок

Подсказка: Подумайте, что измеряет время, что — количество, а что — ресурс.

- **Долгий отклик запроса** — Span с большой задержкой
- **Перегруженный сервис** — Gauge или CPU/Memory метрика
- **Высокий процент ошибок** — Counter ошибок

🔮 Шаг 07:

Миссия 7

Сопоставьте этапы анализа узкой точки с соответствующим инструментом:

Ответ:

- Выявление запроса с максимальной задержкой
 - Сбор распределения времени отклика всех запросов
 - Мониторинг ресурсов сервиса
 - Визуализация зависимости сервисов
1. Span
 2. Histogram
 3. Gauge
 4. Сервисная карта

Подсказка: Подумайте, какой инструмент нужен для индивидуальных запросов, а какой — для агрегатов.

- **Выявление запроса с максимальной задержкой** — Span
- **Сбор распределения времени отклика всех запросов** — Histogram
- **Мониторинг ресурсов сервиса** — Gauge
- **Визуализация зависимости сервисов** — Сервисная карта

🔮 Шаг 08:

Миссия 8

Расположите шаги поиска горячей точки в правильном порядке:

1. Сбор данных: spans и метрики

2. Анализ задержек через spans
3. Анализ нагрузки и ресурсов через метрики
4. Сопоставление узких мест с сервисной картой
5. Выявление и устранение горячих точек

Подсказка: Сначала собираем данные, потом анализируем по разным аспектам, затем визуализируем и исправляем.

Ответ:

- Сбор данных: spans и метрики
- Анализ задержек через spans
- Анализ нагрузки и ресурсов через метрики
- Сопоставление узких мест с сервисной картой
- Выявление и устранение горячих точек

Модуль 6 - Продвинутые практики

Урок 1 - Микросервисы распределённые трассы

Шаг 01:

Миссия 1

Что первым делом стоит отследить, когда микросервис работает медленно?

Подсказка: Думайте о самом медленном участке запроса.

Ответ:

- **Span с максимальной задержкой**
- Общее количество запросов
- Размер лог-файла

Шаг 02:

Миссия 2

Какой инструмент удобно использовать, чтобы увидеть поток запроса через все сервисы?

Подсказка: Это инструмент для распределённой трассировки.

Ответ:

- **Span**
- Gauge
- Counter

Шаг 03:

Миссия 3

Сопоставьте инструмент с его назначением в распределённой трассировке:

Ответ:

- Span
- Histogram метрики времени отклика
- Gauge нагрузки
- Сервисная карта

1. Отслеживание пути запроса и задержек

2. Анализ распределения времени отклика
3. Мониторинг ресурсов в реальном времени
4. Визуализация зависимостей сервисов

Подсказка: Span детализирует один запрос, Histogram агрегирует данные, Gauge показывает ресурсы, карта — связи.

- **Span** — Отслеживание пути запроса и задержек
- **Histogram метрики времени отклика** — Анализ распределения времени отклика
- **Gauge нагрузки** — Мониторинг ресурсов в реальном времени
- **Сервисная карта** — Визуализация зависимостей сервисов

Шаг 04:

Миссия 4

Сопоставьте проблему с подходящей метрикой:

Ответ:

- Долгий отклик запроса
 - Перегруженный сервис
 - Высокий процент ошибок
1. Span с большой задержкой
 2. Gauge CPU/Memory
 3. Counter ошибок

Подсказка: Время, ресурсы и количество ошибок измеряются разными инструментами.

- **Долгий отклик запроса** — Span с большой задержкой
- **Перегруженный сервис** — Gauge CPU/Memory
- **Высокий процент ошибок** — Counter ошибок

Шаг 05:

Миссия 5

Сопоставьте этап анализа узкой точки с инструментом:

Ответ:

- Выявление запроса с максимальной задержкой
 - Сбор распределения времени отклика всех запросов
 - Мониторинг ресурсов сервиса
 - Визуализация цепочки сервисов
1. Span
 2. Histogram
 3. Gauge
 4. Сервисная карта

Подсказка: Локальные детали запроса и агрегированные показатели требуют разных инструментов.

- **Выявление запроса с максимальной задержкой** — Span
- **Сбор распределения времени отклика всех запросов** — Histogram
- **Мониторинг ресурсов сервиса** — Gauge
- **Визуализация цепочки сервисов** — Сервисная карта

🔗 Шаг 06:

Миссия 6

Расположите действия при диагностике медленного микросервиса:

1. Собираем spans для медленных запросов
2. Смотрим распределение времени отклика через гистограммы
3. Анализируем нагрузку на CPU и память
4. Выявляем, где задержка возникает чаще всего
5. Предлагаем оптимизацию

Подсказка: Логика: данные → анализ → выявление → решение.

Ответ:

- Собираем spans для медленных запросов
- Смотрим распределение времени отклика через гистограммы
- Анализируем нагрузку на CPU и память
- Выявляем, где задержка возникает чаще всего
- Предлагаем оптимизацию

📖 Урок 2 - Мониторинг очередей, кэширования, баз данных

🔗 Шаг 01:

Миссия 1

Какой показатель важно отслеживать в очередях сообщений, чтобы понимать, что нагрузка растёт?

Подсказка: Смотрите на то, сколько задач ожидает обработки.

Ответ:

- **Длина очереди**
- Общее количество сообщений за день
- Размер лог-файла очереди

🔗 Шаг 02:

Миссия 2

Какой инструмент удобно использовать для мониторинга кэшей в Python-приложении?

Подсказка: Подумайте, что измеряет количество хитов и промахов.

Ответ:

- **Counter хитов и промахов**
- Gauge использования CPU
- Span времени выполнения запроса

🔗 Шаг 03:

Миссия 3

Какой метрикой удобно отслеживать базу данных при росте нагрузки?

Подсказка: Смотрите на показатели, которые растут при увеличении числа запросов и времени их обработки.

Ответ:

- **Gauge и Histogram для задержек и количества подключений**

- Количество строк в таблице
- Размер лог-файла базы

🔗 Шаг 04:

Миссия 4

Что поможет понять, что кэш неэффективен и требует оптимизации?

Подсказка: Думайте о соотношении попаданий и промахов.

Ответ:

- **Низкий коэффициент хитов кэша**
- Высокое количество логов
- Длинные имена таблиц

🔗 Шаг 05:

Миссия 5

Сопоставьте компонент с метрикой, которая помогает его мониторить:

Ответ:

- Очередь сообщений
 - Кэш
 - База данных
1. Длина очереди и время обработки сообщений
 2. Коэффициент хитов/промахов
 3. Количество подключений и время отклика

Подсказка: Каждая система имеет свои уникальные показатели, отражающие её нагрузку и эффективность.

- **Очередь сообщений** — Длина очереди и время обработки сообщений
- **Кэш** — Коэффициент хитов/промахов
- **База данных** — Количество подключений и время отклика

🔗 Шаг 06:

Миссия 6

Сопоставьте тип нагрузки с подходящей метрикой:

Ответ:

- Растущая очередь задач
 - Увеличение промахов кэша
 - Долгие запросы к базе данных
1. Gauge длины очереди
 2. Counter промахов кэша
 3. Histogram времени отклика запросов

Подсказка: Разные проблемы требуют разных метрик для точного мониторинга.

- **Растущая очередь задач** — Gauge длины очереди
- **Увеличение промахов кэша** — Counter промахов кэша
- **Долгие запросы к базе данных** — Histogram времени отклика запросов

Шаг 07:

Миссия 7

Сопоставьте инструмент OpenTelemetry с его задачей для мониторинга инфраструктуры:

Ответ:

- Span
 - Counter
 - Gauge
 - Histogram
1. Трассировка выполнения запроса или задачи
 2. Счётчик событий (например, промахов кэша)
 3. Измерение текущего состояния ресурса (длина очереди, загрузка CPU)
 4. Анализ распределения времени отклика или задержки задач

Подсказка: Различайте измерение состояния, подсчёт событий и анализ времени выполнения.

- **Span** — Трассировка выполнения запроса или задачи
- **Counter** — Счётчик событий (например, промахов кэша)
- **Gauge** — Измерение текущего состояния ресурса (длина очереди, загрузка CPU)
- **Histogram** — Анализ распределения времени отклика или задержки задач

Шаг 08:

Миссия 8

Расположите шаги мониторинга очереди сообщений:

1. Сбор текущей длины очереди
2. Сбор времени обработки каждой задачи (span)
3. Анализ роста очереди и задержек
4. Выявление узких мест и оптимизация потребителей очереди

Подсказка: Логика: сначала собираем данные, потом анализируем и исправляем.

Ответ:

- Сбор текущей длины очереди
- Сбор времени обработки каждой задачи (span)
- Анализ роста очереди и задержек
- Выявление узких мест и оптимизация потребителей очереди

Шаг 09:

Миссия 9

Расположите шаги мониторинга кэша:

1. Сбор данных о попаданиях и промахах
2. Сравнение коэффициента хитов с ожидаемым
3. Анализ причин промахов
4. Оптимизация стратегии кэширования

Подсказка: Думайте: измеряем → анализируем → исправляем.

Ответ:

- Сбор данных о попаданиях и промахах

- Сравнение коэффициента хитов с ожидаемым
- Анализ причин промахов
- Оптимизация стратегии кэширования

🔧 Шаг 10:

Миссия 10

Расположите шаги мониторинга базы данных под высокой нагрузкой:

1. Сбор текущего числа подключений и времени отклика
2. Сбор распределения времени выполнения запросов (Histogram)
3. Сравнение метрик с пороговыми значениями
4. Выявление медленных запросов и узких мест
5. Оптимизация запросов и проверка улучшений

Подсказка: Логика: собираем данные → анализируем → выявляем проблемные места → исправляем.

Ответ:

- Сбор текущего числа подключений и времени отклика
- Сбор распределения времени выполнения запросов (Histogram)
- Сравнение метрик с пороговыми значениями
- Выявление медленных запросов и узких мест
- Оптимизация запросов и проверка улучшений

📖 Урок 3 - Контроль нагрузок throttling и sampling в OTEL

🔧 Шаг 01:

Миссия 1

Что делает throttling в высоконагруженном приложении?

Подсказка: Подумайте, как приложение контролирует поток запросов.

Ответ:

- **Ограничивает количество запросов в единицу времени**
- Ускоряет все запросы
- Автоматически масштабирует сервер

🔧 Шаг 02:

Миссия 2

Что делает sampling в OpenTelemetry?

Подсказка: Подумайте о том, как уменьшить количество собираемых трасс без потери важных данных.

Ответ:

- **Выбирает часть трасс для отправки и анализа**
- Отправляет все трассы без фильтрации
- Создает новые метрики для всех событий

🔧 Шаг 03:

Миссия 3

Почему в высоконагруженных приложениях важно использовать throttling и sampling?

Подсказка: Думайте о нагрузке и объеме собираемых данных.

Ответ:

- **Чтобы снизить нагрузку на систему и уменьшить объём телеметрии**
- Чтобы ускорить обработку данных на клиенте
- Чтобы автоматически исправлять ошибки

🔮 Шаг 04:

Миссия 4

Какой тип sampling выбирается, если нужно равномерно собирать трассы?

Подсказка: Подумайте о вероятности выбора каждой трассы.

Ответ:

- **Probability sampling**
- Throttling sampling
- Random burst sampling

🔮 Шаг 05:

Миссия 5

Сопоставьте термин с его объяснением:

Ответ:

- Throttling
- Sampling
- Rate Limiting

1. Контроль частоты запросов, чтобы не перегружать систему
2. Выбор части данных для сбора и анализа
3. Ограничение общего количества операций за период времени

Подсказка: Думайте о частоте и объеме обрабатываемых данных.

- **Throttling** — Контроль частоты запросов, чтобы не перегружать систему
- **Sampling** — Выбор части данных для сбора и анализа
- **Rate Limiting** — Ограничение общего количества операций за период времени

🔮 Шаг 06:

Миссия 6

Сопоставьте сценарий с рекомендуемым методом:

Ответ:

- Высокая нагрузка на эндпоинт
- Много мелких запросов для трассировки
- Ограниченные ресурсы хранилища для телеметрии

1. Throttling
2. Sampling
3. Sampling + фильтры

Подсказка: Думайте, что помогает контролировать нагрузку и что уменьшает объём данных.

- **Высокая нагрузка на эндпоинт** — Throttling

- **Много мелких запросов для трассировки** — Sampling
- **Ограниченные ресурсы хранилища для телеметрии** — Sampling + фильтры

Шаг 07:

Миссия 7

Сопоставьте вид throttling с его примером:

Ответ:

- Token Bucket
- Leaky Bucket
- Fixed Window

1. Позволяет временно превышать лимит, используя накопленные токены
2. Постоянный поток без резких пиков
3. Ограничение запросов в фиксированный период времени

Подсказка: Разные алгоритмы управления потоком имеют разные правила.

- **Token Bucket** — Позволяет временно превышать лимит, используя накопленные токены
- **Leaky Bucket** — Постоянный поток без резких пиков
- **Fixed Window** — Ограничение запросов в фиксированный период времени

Шаг 08:

Миссия 8

Расположите шаги настройки sampling для микросервиса:

1. Определение цели трассировки (какие сервисы важны)
2. Выбор стратегии sampling (probability, rate-limiting)
3. Настройка фильтров и порогов
4. Внедрение в код или middleware
5. Мониторинг и корректировка правил sampling

Подсказка: Думайте логически: сначала цель, потом стратегия, потом реализация и контроль.

Ответ:

- Определение цели трассировки (какие сервисы важны)
- Выбор стратегии sampling (probability, rate-limiting)
- Настройка фильтров и порогов
- Внедрение в код или middleware
- Мониторинг и корректировка правил sampling

Шаг 09:

Миссия 9

Расположите шаги применения throttling на высоконагруженном эндпоинте:

1. Анализ текущей нагрузки и пиков
2. Выбор алгоритма throttling (Token Bucket, Leaky Bucket, Fixed Window)
3. Настройка лимитов и временных окон
4. Внедрение контроля на уровне сервиса
5. Мониторинг эффективности и корректировка параметров

Подсказка: Думайте последовательно: анализ → выбор → настройка → внедрение → проверка.

Ответ:

- Анализ текущей нагрузки и пиков
- Выбор алгоритма throttling (Token Bucket, Leaky Bucket, Fixed Window)
- Настройка лимитов и временных окон
- Внедрение контроля на уровне сервиса
- Мониторинг эффективности и корректировка параметров

Урок 4 - Практика настройка pipeline для высоконагруженного сервиса

Шаг 01:

Миссия 1

Что является первой задачей при настройке pipeline для высоконагруженного сервиса?

Подсказка: Сначала нужно понять, какие данные и события мы будем обрабатывать.

Ответ:

- **Определение источников данных и событий**
- Настройка мониторинга метрик
- Запуск тестов нагрузки

Шаг 02:

Миссия 2

Для чего важно использовать batching при высоких нагрузках?

Подсказка: Подумайте о том, как уменьшить количество операций и ускорить обработку.

Ответ:

- **Чтобы объединять события и отправлять их пакетами для эффективности**
- Чтобы проверять каждое событие отдельно
- Чтобы уменьшить количество серверов

Шаг 03:

Миссия 3

Что такое backpressure в pipeline?

Подсказка: Думайте о контроле потока данных, чтобы избежать перегрузки.

Ответ:

- **Механизм замедления или приостановки поступления данных при перегрузке**
- Удаление старых данных из pipeline
- Автоматическое масштабирование сервисов

Шаг 04:

Миссия 4

Почему важно логировать ошибки в pipeline?

Подсказка: Думайте о том, как отследить, где и почему произошел сбой.

Ответ:

- **Чтобы иметь возможность анализировать и исправлять проблемы**
- Чтобы ускорить обработку данных
- Чтобы уменьшить нагрузку на сервис

🔧 Шаг 05:

Миссия 5

Сопоставьте элементы pipeline с их ролью:

Ответ:

- Collector
 - Processor
 - Exporter
1. Собирает данные из разных источников
 2. Обрабатывает, фильтрует или агрегирует события
 3. Отправляет данные в хранилище или систему мониторинга

Подсказка: Думайте о цепочке: сбор → обработка → отправка.

- **Collector** — Собирает данные из разных источников
- **Processor** — Обрабатывает, фильтрует или агрегирует события
- **Exporter** — Отправляет данные в хранилище или систему мониторинга

🔧 Шаг 06:

Миссия 6

Сопоставьте стратегию с её преимуществом:

Ответ:

- Batching
 - Throttling
 - Sampling
1. Снижает нагрузку на систему, отправляя события порциями
 2. Контролирует поток данных, чтобы не перегружать сервис
 3. Снижает объём телеметрии, сохраняя ключевые трассы

Подсказка: Каждая стратегия решает конкретную проблему с нагрузкой или объёмом данных.

- **Batching** — Снижает нагрузку на систему, отправляя события порциями
- **Throttling** — Контролирует поток данных, чтобы не перегружать сервис
- **Sampling** — Снижает объём телеметрии, сохраняя ключевые трассы

🔧 Шаг 07:

Миссия 7

Сопоставьте тип метрик с примером:

Ответ:

- Счётчик (Counter)
 - Гейдж (Gauge)
 - Гистограмма (Histogram)
1. Общее количество обработанных запросов
 2. Текущее количество активных соединений
 3. Распределение времени обработки запросов

Подсказка: Разные метрики измеряют разные аспекты сервиса.

- **Счётчик (Counter)** — Общее количество обработанных запросов
- **Гейдж (Gauge)** — Текущее количество активных соединений
- **Гистограмма (Histogram)** — Распределение времени обработки запросов

🔧 Шаг 08:

Миссия 8

Расположите шаги настройки pipeline в правильном порядке:

1. Определение источников данных
2. Настройка Collector
3. Настройка Processor
4. Настройка Exporter
5. Тестирование и мониторинг

Подсказка: Думайте логично: сначала собираем, потом обрабатываем, потом отправляем и проверяем.

Ответ:

- Определение источников данных
- Настройка Collector
- Настройка Processor
- Настройка Exporter
- Тестирование и мониторинг

🔧 Шаг 09:

Миссия 9

Расположите этапы оптимизации pipeline для высокой нагрузки:

1. Внедрение batching
2. Применение throttling
3. Настройка sampling
4. Мониторинг производительности
5. Корректировка параметров

Подсказка: Подумайте, как постепенно улучшать производительность и стабильность.

Ответ:

- Внедрение batching
- Применение throttling
- Настройка sampling
- Мониторинг производительности
- Корректировка параметров

🔧 Шаг 10:

Миссия 10

Расположите этапы комплексного развертывания pipeline для высоконагруженного сервиса:

1. Определение источников и целей данных
2. Настройка Collector, Processor, Exporter
3. Применение стратегий batching, throttling, sampling
4. Интеграция с мониторингом и алертингом

5. Тестирование и оптимизация работы сервиса

Подсказка: Думайте, как строится полный цикл: планирование → настройка → контроль → оптимизация.

Ответ:

- Определение источников и целей данных
- Настройка Collector, Processor, Exporter
- Применение стратегий batching, throttling, sampling
- Интеграция с мониторингом и алертингом
- Тестирование и оптимизация работы сервиса

Урок 5 - Кейсы ошибки, аварии и восстановление

Шаг 01:

Миссия 1

Что является первым шагом при обнаружении аварии в высоконагруженном сервисе?

Подсказка: Сначала нужно понять, где именно произошёл сбой, прежде чем что-то исправлять.

Ответ:

- **Выявление источника ошибки и зоны сбоя**
- Немедленное перезапуск сервиса
- Изменение кода без анализа

Шаг 02:

Миссия 2

Почему важно собирать трассы и логи при авариях?

Подсказка: Подумайте, как инженеры находят причину проблем в сложных системах.

Ответ:

- **Чтобы понять последовательность событий и локализовать проблему**
- Чтобы увеличить нагрузку на систему
- Чтобы автоматически исправлять ошибки без анализа

Шаг 03:

Миссия 3

Что является хорошей практикой для минимизации потерь при сбое очереди сообщений?

Подсказка: Думайте о гарантированной доставке данных и резервных механизмах.

Ответ:

- **Использование персистентных очередей и повторных попыток**
- Игнорирование сообщений при перегрузке
- Удаление старых сообщений сразу

Шаг 04:

Миссия 4

Какой инструмент OpenTelemetry помогает быстро обнаруживать аномалии в сервисе?

Подсказка: Подумайте о способах мониторинга и визуализации метрик.

Ответ:

- **Трассировка и метрики с алертингом**
- Удаление логов
- Игнорирование ошибок

Шаг 05:

Миссия 5

Сопоставьте тип ошибки с примером её проявления в сервисе:

Ответ:

- Network error
 - Timeout
 - Data corruption
1. Пакеты не дошли до сервера или потеря соединения
 2. Ответ от сервера занимает слишком много времени
 3. Данные не соответствуют ожидаемому формату или повреждены

Подсказка: Думайте, как каждая проблема проявляется с точки зрения клиента или сервиса.

- **Network error** — Пакеты не дошли до сервера или потеря соединения
- **Timeout** — Ответ от сервера занимает слишком много времени
- **Data corruption** — Данные не соответствуют ожидаемому формату или повреждены

Шаг 06:

Миссия 6

Сопоставьте стратегию восстановления с её описанием:

Ответ:

- Retry
 - Circuit breaker
 - Fallback
1. Повторная попытка выполнения операции при временной ошибке
 2. Прекращение вызовов к сервису при превышении ошибок, чтобы защитить систему
 3. Использование альтернативного ресурса при сбое основного

Подсказка: Каждая стратегия решает разные типы сбоев.

- **Retry** — Повторная попытка выполнения операции при временной ошибке
- **Circuit breaker** — Прекращение вызовов к сервису при превышении ошибок, чтобы защитить систему
- **Fallback** — Использование альтернативного ресурса при сбое основного

Шаг 07:

Миссия 7

Сопоставьте тип мониторинга с примером:

Ответ:

- Logs
- Metrics

- Traces

1. Подробная информация о событиях и ошибках в сервисе
2. Количество запросов, среднее время ответа
3. Последовательность вызовов между микросервисами

Подсказка: Разные типы телеметрии дают разный уровень понимания происходящего.

- **Logs** — Подробная информация о событиях и ошибках в сервисе
- **Metrics** — Количество запросов, среднее время ответа
- **Traces** — Последовательность вызовов между микросервисами

Шаг 08:

Миссия 8

Расположите шаги восстановления сервиса после критической ошибки:

1. Обнаружение и локализация ошибки
2. Применение стратегии восстановления (retry, fallback, circuit breaker)
3. Анализ причин аварии
4. Мониторинг и проверка корректной работы
5. Документирование и улучшение процессов

Подсказка: Думайте логично: сначала находим проблему, потом исправляем, потом анализируем и улучшаем.

Ответ:

- Обнаружение и локализация ошибки
- Применение стратегии восстановления (retry, fallback, circuit breaker)
- Анализ причин аварии
- Мониторинг и проверка корректной работы
- Документирование и улучшение процессов

Шаг 09:

Миссия 9

Расположите шаги реагирования на cascade failure в микросервисах:

1. Идентификация зависимых сервисов
2. Применение circuit breaker для защиты системы
3. Ограничение входящего трафика или throttling
4. Восстановление критически важных сервисов
5. Анализ причин и улучшение архитектуры

Подсказка: Сначала защищаем систему, потом восстанавливаем и анализируем.

Ответ:

- Идентификация зависимых сервисов
- Применение circuit breaker для защиты системы
- Ограничение входящего трафика или throttling
- Восстановление критически важных сервисов
- Анализ причин и улучшение архитектуры

Модуль 7 - Практический проект

Урок 1 - Создание простого HTTP-сервиса с микросервисной архитектурой

Шаг 01:

Миссия 1

Что является первым шагом при создании простого HTTP-сервиса?

Подсказка: Подумайте, что нужно сделать до того, как писать код.

Ответ:

- **Определение требований и API сервиса**
- Сразу писать все микросервисы
- Игнорировать архитектуру

Шаг 02:

Миссия 2

Почему важно использовать микросервисную архитектуру для масштабируемого сервиса?

Подсказка: Подумайте, как сервис может расти без простоя.

Ответ:

- **Чтобы можно было масштабировать отдельные части независимо**
- Чтобы весь сервис был монолитным
- Чтобы усложнить разработку без пользы

Шаг 03:

Миссия 3

Что является ключевым при организации взаимодействия между микросервисами?

Подсказка: Подумайте, как сервисы общаются друг с другом и обрабатывают ошибки.

Ответ:

- **Использование HTTP API и надёжных протоколов обмена данными**
- Прямое подключение к базе каждого сервиса
- Игнорирование ошибок при вызове другого сервиса

Шаг 04:

Миссия 4

Почему важно логировать запросы и ответы в HTTP-сервисе?

Подсказка: Подумайте, как можно понять работу сервиса и отладить ошибки.

Ответ:

- **Чтобы отслеживать работу сервиса и выявлять проблемы**
- Чтобы увеличивать нагрузку на сервис без пользы
- Чтобы скрывать ошибки от команды

Шаг 05:

Миссия 5

Сопоставьте компонент микросервисной архитектуры с его функцией:

Ответ:

- API Gateway
 - Service
 - Database
 - Monitoring
1. Обеспечивает единый вход и маршрутизацию запросов
 2. Обработывает бизнес-логику
 3. Хранит данные и обеспечивает доступ к ним
 4. Собирает метрики и логи для анализа

Подсказка: Каждый компонент решает конкретную задачу в архитектуре.

- **API Gateway** — Обеспечивает единый вход и маршрутизацию запросов
- **Service** — Обработывает бизнес-логику
- **Database** — Хранит данные и обеспечивает доступ к ним
- **Monitoring** — Собирает метрики и логи для анализа

 Шаг 06:

Миссия 6

Сопоставьте тип запроса с его использованием в HTTP-сервисе:

Ответ:

- GET
 - POST
 - PUT
 - DELETE
1. Получение данных без изменения состояния
 2. Создание новых ресурсов
 3. Обновление существующих ресурсов
 4. Удаление ресурсов

Подсказка: Думайте о CRUD-операциях и HTTP-методах.

- **GET** — Получение данных без изменения состояния
- **POST** — Создание новых ресурсов
- **PUT** — Обновление существующих ресурсов
- **DELETE** — Удаление ресурсов

 Шаг 07:

Миссия 7

Сопоставьте термин OpenTelemetry с его ролью в сервисе:

Ответ:

- Traces
 - Metrics
 - Logs
1. Отслеживает последовательность вызовов между сервисами
 2. Измеряет показатели производительности, нагрузку и ошибки
 3. Содержит текстовую информацию о событиях и ошибках

Подсказка: Каждая телеметрия помогает наблюдать за сервисом с разной стороны.

- **Traces** — Отслеживает последовательность вызовов между сервисами
- **Metrics** — Измеряет показатели производительности, нагрузку и ошибки
- **Logs** — Содержит текстовую информацию о событиях и ошибках

Шаг 08:

Миссия 8

Расположите шаги запуска HTTP-сервиса в правильном порядке:

1. Создание виртуального окружения и установка зависимостей
2. Разработка микросервисов и API
3. Настройка базы данных и внешних сервисов
4. Запуск сервиса и тестирование API
5. Настройка мониторинга и логирования

Подсказка: Думайте о порядке от подготовки кода до наблюдения за сервисом в работе.

Ответ:

- Создание виртуального окружения и установка зависимостей
- Разработка микросервисов и API
- Настройка базы данных и внешних сервисов
- Запуск сервиса и тестирование API
- Настройка мониторинга и логирования

Шаг 09:

Миссия 9

Расположите этапы масштабирования микросервисной архитектуры:

1. Мониторинг производительности и нагрузки
2. Определение узких мест
3. Горизонтальное масштабирование сервисов
4. Настройка балансировки нагрузки
5. Тестирование и контроль стабильности

Подсказка: Думайте о логике: сначала наблюдаем, затем масштабируем, затем проверяем стабильность.

Ответ:

- Мониторинг производительности и нагрузки
- Определение узких мест
- Горизонтальное масштабирование сервисов
- Настройка балансировки нагрузки
- Тестирование и контроль стабильности

Урок 2 - Подключение OTEL middleware, spans, атрибуты

Шаг 01:

Миссия 1

Что является первым шагом при подключении OpenTelemetry к Python-сервису?

Подсказка: Думайте о базовой настройке библиотеки и инициализации трассировки.

Ответ:

- **Инициализация OTEL SDK и настройка экспортеров**
- Сразу создавать spans вручную
- Игнорировать конфигурацию SDK

Шаг 02:

Миссия 2

Почему важно подключать middleware для HTTP-сервиса в OTEL?

Подсказка: Думайте, как автоматическая трассировка помогает наблюдать за всеми запросами.

Ответ:

- **Чтобы автоматически создавать spans для входящих запросов**
- Чтобы усложнить код сервиса
- Чтобы полностью отключить логику обработки запросов

Шаг 03:

Миссия 3

Что такое span в OpenTelemetry?

Подсказка: Подумайте о «отрезках» работы внутри сервиса, которые мы хотим измерять.

Ответ:

- **Отдельный элемент трассировки, представляющий работу функции или запроса**
- Полный лог всех операций сервиса
- Библиотека для мониторинга CPU

Шаг 04:

Миссия 4

Почему важно добавлять атрибуты к span?

Подсказка: Подумайте, как дополнительная информация помогает анализировать поведение сервиса.

Ответ:

- **Чтобы добавить контекст к span и легче понимать причины задержек**
- Чтобы уменьшить объём трассировки
- Чтобы автоматически исправлять ошибки

Шаг 05:

Миссия 5

Сопоставьте компонент OTEL с его ролью:

Ответ:

- Tracer
- Span
- Exporter
- Attributes

1. Создаёт и управляет span-ами
2. Единичная запись о выполнении операции
3. Отправляет данные в систему наблюдения

4. Добавляет дополнительную информацию к span

Подсказка: Каждый элемент OTEL выполняет конкретную функцию в трассировке.

- **Tracer** — Создаёт и управляет span-ами
- **Span** — Единичная запись о выполнении операции
- **Exporter** — Отправляет данные в систему наблюдения
- **Attributes** — Добавляет дополнительную информацию к span

Шаг 06:

Миссия 6

Сопоставьте middleware с его назначением в OTEL:

Ответ:

- HTTP middleware
 - Database middleware
 - Custom middleware
1. Автоматическая трассировка входящих HTTP-запросов
 2. Отслеживание запросов к базе данных
 3. Трассировка специфических операций сервиса

Подсказка: Middleware подключается к разным слоям системы для наблюдения.

- **HTTP middleware** — Автоматическая трассировка входящих HTTP-запросов
- **Database middleware** — Отслеживание запросов к базе данных
- **Custom middleware** — Трассировка специфических операций сервиса

Шаг 07:

Миссия 7

Сопоставьте тип span с примером использования:

Ответ:

- Server span
 - Client span
 - Internal span
1. Отслеживает обработку входящего запроса на сервере
 2. Отслеживает исходящий запрос к другому сервису
 3. Отслеживает внутреннюю функцию внутри сервиса

Подсказка: Подумайте о направлении и контексте span.

- **Server span** — Отслеживает обработку входящего запроса на сервере
- **Client span** — Отслеживает исходящий запрос к другому сервису
- **Internal span** — Отслеживает внутреннюю функцию внутри сервиса

Шаг 08:

Миссия 8

Расположите шаги создания span с атрибутами в правильном порядке:

1. Инициализация tracer
2. Создание нового span для операции

3. Добавление атрибутов к span
4. Выполнение кода операции
5. Завершение span

Подсказка: Думайте логично: сначала подготовка, потом создание, добавление данных, выполнение, завершение.

Ответ:

- Инициализация tracer
- Создание нового span для операции
- Добавление атрибутов к span
- Выполнение кода операции
- Завершение span

Шаг 09:

Миссия 9

Расположите этапы подключения OTEL middleware к HTTP-сервису:

1. Установка OTEL SDK и зависимостей
2. Импорт middleware в приложение
3. Инициализация tracer и exporter
4. Подключение middleware к HTTP серверу
5. Тестирование и проверка span-ов

Подсказка: Сначала готовим среду, затем интегрируем middleware и проверяем работу.

Ответ:

- Установка OTEL SDK и зависимостей
- Импорт middleware в приложение
- Инициализация tracer и exporter
- Подключение middleware к HTTP серверу
- Тестирование и проверка span-ов

Шаг 10:

Миссия 10

Расположите шаги комплексного наблюдения с OTEL в сервисе:

1. Подключение tracer и middleware
2. Создание span-ов с атрибутами для критических операций
3. Сбор метрик, логов и трассировок
4. Экспорт данных в систему наблюдения
5. Анализ и оптимизация работы сервиса

Подсказка: Думайте о цикле наблюдения: от подключения до анализа и улучшения.

Ответ:

- Подключение tracer и middleware
- Создание span-ов с атрибутами для критических операций
- Сбор метрик, логов и трассировок
- Экспорт данных в систему наблюдения
- Анализ и оптимизация работы сервиса

Урок 3 - Настройка Batch Exporter и Collector

Шаг 01:

Миссия 1

Что делает Batch Exporter в OpenTelemetry?

Подсказка: Подумайте о том, как данные собираются и отправляются пакетами.

Ответ:

- **Собирает spans и отправляет их пакетами для повышения производительности**
- Отправляет каждый span сразу, без пакета
- Удаляет старые spans после экспорта

Шаг 02:

Миссия 2

Почему использование Collector полезно в высоконагруженном сервисе?

Подсказка: Подумайте, как централизованная обработка помогает разгружать сервисы.

Ответ:

- **Позволяет централизованно собирать, обрабатывать и экспортировать телеметрию**
- Служит для хранения кода приложения
- Снижает точность трассировок

Шаг 03:

Миссия 3

Какая настройка Batch Exporter влияет на частоту отправки данных?

Подсказка: Подумайте о параметрах интервала и размера пакета.

Ответ:

- **Параметры batch_size и schedule_delay**
- Цвет интерфейса приложения
- Версия Python

Шаг 04:

Миссия 4

Как лучше всего использовать Collector для нескольких микросервисов?

Подсказка: Подумайте о единой точке сбора данных.

Ответ:

- **Подключить все микросервисы к одному экземпляру Collector**
- Каждому сервису заводить свой Collector без синхронизации
- Отправлять данные напрямую в базу, минуя Collector

Шаг 05:

Миссия 5

Сопоставьте термин с его ролью в Batch Exporter и Collector:

Ответ:

- Batch Exporter
- Collector
- Schedule Delay
- Batch Size

1. Собирает и отправляет spans пакетами
2. Централизованная обработка телеметрии
3. Задержка между отправками пакетов
4. Максимальное количество span-ов в пакете

Подсказка: Думайте о роли каждого элемента в процессах сбора и передачи данных.

- **Batch Exporter** — Собирает и отправляет spans пакетами
- **Collector** — Централизованная обработка телеметрии
- **Schedule Delay** — Задержка между отправками пакетов
- **Batch Size** — Максимальное количество span-ов в пакете

 Шаг 06:

Миссия 6

Сопоставьте параметры Batch Exporter с эффектом на производительность:

Ответ:

- Большой batch_size
- Малый batch_size
- Короткий schedule_delay
- Длинный schedule_delay

1. Больше нагрузка на сеть, меньше задержка
2. Меньше нагрузка на сеть, больше количество отправок
3. Быстрая отправка пакетов, высокая нагрузка на сеть
4. Меньше частота отправки, экономия ресурсов

Подсказка: Думайте о компромиссе между скоростью передачи и нагрузкой на сеть.

- **Большой batch_size** — Больше нагрузка на сеть, меньше задержка
- **Малый batch_size** — Меньше нагрузка на сеть, больше количество отправок
- **Короткий schedule_delay** — Быстрая отправка пакетов, высокая нагрузка на сеть
- **Длинный schedule_delay** — Меньше частота отправки, экономия ресурсов

 Шаг 07:

Миссия 7

Сопоставьте компонент Collector с его функцией:

Ответ:

- Receiver
- Processor
- Exporter

1. Принимает данные от SDK или других источников
2. Обработывает и фильтрует телеметрию
3. Отправляет данные в целевую систему мониторинга

Подсказка: Collector делится на логические блоки для упрощения работы с данными.

- **Receiver** — Принимает данные от SDK или других источников
- **Processor** — Обрабатывает и фильтрует телеметрию
- **Exporter** — Отправляет данные в целевую систему мониторинга

Шаг 08:

Миссия 8

Расположите шаги настройки Batch Exporter с Collector:

1. Установить OTEL SDK и Collector
2. Настроить exporter в SDK
3. Запустить Collector с конфигурацией
4. Настроить batch_size и schedule_delay
5. Проверить передачу данных

Подсказка: Думайте логично: сначала установка, потом настройка, затем запуск и проверка.

Ответ:

- Установить OTEL SDK и Collector
- Настроить exporter в SDK
- Запустить Collector с конфигурацией
- Настроить batch_size и schedule_delay
- Проверить передачу данных

Шаг 09:

Миссия 9

Расположите шаги интеграции Batch Exporter для нескольких микросервисов:

1. Подключение OTEL SDK к каждому микросервису
2. Настройка Batch Exporter с общими параметрами
3. Подключение всех микросервисов к Collector
4. Запуск и мониторинг передачи пакетов
5. Анализ полученной телеметрии

Подсказка: Сначала подключаем SDK, потом объединяем данные через Collector и анализируем результат.

Ответ:

- Подключение OTEL SDK к каждому микросервису
- Настройка Batch Exporter с общими параметрами
- Подключение всех микросервисов к Collector
- Запуск и мониторинг передачи пакетов
- Анализ полученной телеметрии

Урок 4 - Визуализация трасс в Jaeger Grafana

Шаг 01:

Миссия 1

Что такое Jaeger в контексте OpenTelemetry?

Подсказка: Подумайте о том, как разработчики видят трассы запросов.

Ответ:

- Система визуализации и анализа распределённых трасс
- Библиотека для логирования Python
- Инструмент для управления базой данных

Шаг 02:

Миссия 2

Для чего используется Grafana с OpenTelemetry?

Подсказка: Подумайте о визуализации метрик и графиков для наблюдения за системой.

Ответ:

- Для создания интерактивных дашбордов с метриками и трассами
- Для хранения данных приложений
- Для запуска микросервисов

Шаг 03:

Миссия 3

Что позволяет делать просмотр трасс в Jaeger?

Подсказка: Думайте о том, как отслеживаются последовательности вызовов функций.

Ответ:

- Анализировать путь запроса через все микросервисы
- Редактировать код микросервиса
- Оптимизировать базу данных автоматически

Шаг 04:

Миссия 4

Что такое span в визуализации трасс?

Подсказка: Думайте о маленьком шаге внутри большого пути запроса.

Ответ:

- Единица работы, представляющая один шаг запроса или операции
- Общее количество пользователей сервиса
- Конфигурация базы данных

Шаг 05:

Миссия 5

Сопоставьте элементы трасс с их описанием:

Ответ:

- Trace
 - Span
 - Tag
 - Log
1. Цепочка вызовов одного запроса через сервисы
 2. Отдельная операция или шаг запроса
 3. Метаданные о span-е
 4. Запись события внутри span-а

Подсказка: Каждый элемент отражает часть наблюдаемого запроса.

- **Trace** — Цепочка вызовов одного запроса через сервисы
- **Span** — Отдельная операция или шаг запроса
- **Tag** — Метаданные о span-е
- **Log** — Запись события внутри span-а

🔧 Шаг 06:

Миссия 6

Сопоставьте визуализацию в Jaeger с её функцией:

Ответ:

- Timeline
 - Service Graph
 - Dependency Diagram
1. Показывает последовательность span-ов по времени
 2. Визуализирует взаимодействие между сервисами в одном trace
 3. Показывает зависимости всех сервисов системы

Подсказка: Подумайте о разных способах отображения трасс.

- **Timeline** — Показывает последовательность span-ов по времени
- **Service Graph** — Визуализирует взаимодействие между сервисами в одном trace
- **Dependency Diagram** — Показывает зависимости всех сервисов системы

🔧 Шаг 07:

Миссия 7

Сопоставьте элементы Grafana с их назначением:

Ответ:

- Dashboard
 - Panel
 - Alert
 - Data Source
1. Место, где отображаются графики и метрики
 2. Отдельный график или таблица на dashboard-е
 3. Настройка уведомлений при превышении порогов
 4. Источник данных для отображения

Подсказка: Каждый элемент отвечает за визуализацию, данные или оповещения.

- **Dashboard** — Место, где отображаются графики и метрики
- **Panel** — Отдельный график или таблица на dashboard-е
- **Alert** — Настройка уведомлений при превышении порогов
- **Data Source** — Источник данных для отображения

🔧 Шаг 08:

Миссия 8

Расположите шаги просмотра трассы в Jaeger:

1. Выбор сервиса и trace ID

2. Открытие timeline span-ов
3. Анализ зависимостей через Service Graph
4. Проверка логов и тегов span-ов

Подсказка: Думайте о логической последовательности анализа запроса.

Ответ:

- Выбор сервиса и trace ID
- Открытие timeline span-ов
- Анализ зависимостей через Service Graph
- Проверка логов и тегов span-ов

 Шаг 09:

Миссия 9

Расположите шаги создания дашборда в Grafana для трасс OpenTelemetry:

1. Добавление Data Source с телеметрией
2. Создание Dashboard
3. Добавление Panels для визуализации метрик и span-ов
4. Настройка Alerts при превышении порогов
5. Сохранение и проверка Dashboard

Подсказка: Логика: сначала источник данных, затем визуализация и оповещения.

Ответ:

- Добавление Data Source с телеметрией
- Создание Dashboard
- Добавление Panels для визуализации метрик и span-ов
- Настройка Alerts при превышении порогов
- Сохранение и проверка Dashboard

 Шаг 10:

Миссия 10

Расположите шаги анализа производительности микросервисов через Jaeger и Grafana:

1. Выбор интересующего trace-а или метрики
2. Открытие Timeline и анализ span-ов
3. Использование Service Graph для поиска узких мест
4. Сравнение метрик на дашборде Grafana
5. Документирование проблем и оптимизация кода

Подсказка: Думайте как инженер-детектив: сначала сбор данных, затем анализ, потом действия.

Ответ:

- Выбор интересующего trace-а или метрики
- Открытие Timeline и анализ span-ов
- Использование Service Graph для поиска узких мест
- Сравнение метрик на дашборде Grafana
- Документирование проблем и оптимизация кода

Урок 5 - Анализ узких мест и оптимизация производительности

Шаг 01:

Миссия 1

Что такое узкое место (bottleneck) в приложении?

Подсказка: Думайте о том, что замедляет работу всей системы.

Ответ:

- **Часть системы, которая ограничивает общую производительность**
- Любая ошибка в коде
- Случайная задержка в сети

Шаг 02:

Миссия 2

Как OpenTelemetry помогает находить узкие места?

Подсказка: Подумайте о том, что собирает информацию о каждом шаге запроса.

Ответ:

- **Собирает трассы и метрики, позволяя видеть задержки и узкие места**
- Автоматически исправляет узкие места
- Создает резервные копии данных

Шаг 03:

Миссия 3

Что чаще всего является узким местом в микросервисном приложении?

Подсказка: Думайте о внешних сервисах и медленных операциях.

Ответ:

- **Медленные запросы к базе данных или внешним API**
- Имена переменных в коде
- Цвет интерфейса пользователя

Шаг 04:

Миссия 4

Почему важно измерять время выполнения span-ов в трассах?

Подсказка: Подумайте, как это помогает найти медленные части запроса.

Ответ:

- **Чтобы идентифицировать узкие места и медленные операции**
- Чтобы уменьшить размер кода
- Чтобы проверить орфографию комментариев

Шаг 05:

Миссия 5

Сопоставьте термины анализа производительности с их назначением:

Ответ:

- Trace

- Span
- Latency
- Throughput

1. Полный путь запроса через все сервисы
2. Отдельная операция или шаг запроса
3. Время выполнения операции
4. Количество обработанных запросов за единицу времени

Подсказка: Думайте, что измеряется для оптимизации.

- **Trace** — Полный путь запроса через все сервисы
- **Span** — Отдельная операция или шаг запроса
- **Latency** — Время выполнения операции
- **Throughput** — Количество обработанных запросов за единицу времени

Шаг 06:

Миссия 6

Сопоставьте методы оптимизации с их эффектом:

Ответ:

- Кэширование
 - Асинхронная обработка
 - Пул соединений
 - Оптимизация запросов к БД
1. Снижение повторной нагрузки на сервисы
 2. Позволяет не блокировать выполнение при ожидании ответа
 3. Сокращает накладные расходы на создание новых соединений
 4. Уменьшение времени выполнения запросов

Подсказка: Думайте о способах ускорить работу узких мест.

- **Кэширование** — Снижение повторной нагрузки на сервисы
- **Асинхронная обработка** — Позволяет не блокировать выполнение при ожидании ответа
- **Пул соединений** — Сокращает накладные расходы на создание новых соединений
- **Оптимизация запросов к БД** — Уменьшение времени выполнения запросов

Шаг 07:

Миссия 7

Сопоставьте инструменты мониторинга с их назначением:

Ответ:

- Jaeger
 - Grafana
 - Prometheus
1. Слежение за распределёнными трассами
 2. Визуализация метрик и дашбордов
 3. Сбор и хранение метрик с возможностью запросов

Подсказка: Думайте, что делает каждый инструмент для анализа производительности.

- **Jaeger** — Слежение за распределёнными трассами

- **Grafana** — Визуализация метрик и дашбордов
- **Prometheus** — Сбор и хранение метрик с возможностью запросов

Шаг 08:

Миссия 8

Расположите шаги анализа узких мест в логическом порядке:

1. Сбор трасс и метрик
2. Определение подозрительных span-ов и операций
3. Анализ латентности и throughput
4. Выявление узких мест
5. Оптимизация кода или конфигурации

Подсказка: Сначала собираем данные, потом анализируем и улучшаем.

Ответ:

- Сбор трасс и метрик
- Определение подозрительных span-ов и операций
- Анализ латентности и throughput
- Выявление узких мест
- Оптимизация кода или конфигурации

Шаг 09:

Миссия 9

Расположите шаги оптимизации запросов к базе данных:

1. Анализ медленных запросов через трассы
2. Добавление индексов там, где это необходимо
3. Переписывание сложных запросов на более эффективные
4. Тестирование и проверка времени выполнения
5. Мониторинг после изменений

Подсказка: Сначала находим проблемные запросы, потом улучшаем их и проверяем эффект.

Ответ:

- Анализ медленных запросов через трассы
- Добавление индексов там, где это необходимо
- Переписывание сложных запросов на более эффективные
- Тестирование и проверка времени выполнения
- Мониторинг после изменений

Шаг 10:

Миссия 10

Расположите этапы комплексной оптимизации производительности микросервисов:

1. Сбор трасс и метрик с OpenTelemetry
2. Анализ узких мест в микросервисах
3. Оптимизация кода, запросов и конфигураций
4. Тестирование производительности после изменений
5. Документирование изменений и мониторинг в реальном времени

Подсказка: Думайте как инженер-детектив: сначала собираем данные, потом анализ, потом исправляем и проверяем.

Ответ:

- Сбор трасс и метрик с OpenTelemetry
- Анализ узких мест в микросервисах
- Оптимизация кода, запросов и конфигураций
- Тестирование производительности после изменений
- Документирование изменений и мониторинг в реальном времени

Модуль 8 - Резюме и рекомендации

Урок 1 - Лучшие практики OTEL в high-load системах

Шаг 01:

Миссия 1

Что из перечисленного является хорошей практикой для high-load систем с OpenTelemetry?

Подсказка: Подумайте, как уменьшить нагрузку на систему при сборе данных.

Ответ:

- **Использовать Batch Exporter для отправки данных пакетами**
- Собирать все span-ы синхронно при каждом запросе
- Игнорировать настройки sampling и отправлять всё подряд

Шаг 02:

Миссия 2

Почему рекомендуется использовать sampling в high-load приложениях?

Подсказка: Подумайте, как уменьшить объём телеметрии без потери полезной информации.

Ответ:

- **Чтобы собирать только часть span-ов и снижать нагрузку на collector**
- Чтобы полностью отключить сбор метрик
- Чтобы ускорить тестирование интерфейсов пользователя

Шаг 03:

Миссия 3

Что важно учитывать при интеграции OTEL в микросервисную архитектуру?

Подсказка: Подумайте о трассировке сквозных запросов через несколько сервисов.

Ответ:

- **Сохранять контекст трассы при межсервисных вызовах**
- Игнорировать распределённые span-ы
- Использовать только метрики без трассировки

Шаг 04:

Миссия 4

Какая практика помогает минимизировать влияние OTEL на производительность приложения?

Подсказка: Думайте о том, как снизить накладные расходы на сбор данных.

Ответ:

- **Использовать асинхронные экспортеры и batch-сборку данных**
- Собирать span-ы по одному синхронно
- Выключать логирование и метрики полностью

Шаг 05:

Миссия 5

Сопоставьте лучшие практики OTEL с их назначением:

Ответ:

- Batch Exporter
 - Sampling
 - Асинхронная отправка
 - Сохранение контекста
1. Снижение нагрузки на collector и сеть
 2. Сбор только части данных для уменьшения объёма
 3. Не блокировать основной поток приложения при отправке данных
 4. Корректная сквозная трассировка между сервисами

Подсказка: Думайте, какая практика влияет на нагрузку, какая — на точность данных.

- **Batch Exporter** — Снижение нагрузки на collector и сеть
- **Sampling** — Сбор только части данных для уменьшения объёма
- **Асинхронная отправка** — Не блокировать основной поток приложения при отправке данных
- **Сохранение контекста** — Корректная сквозная трассировка между сервисами

Шаг 06:

Миссия 6

Сопоставьте подходы к high-load системам с их преимуществами:

Ответ:

- Гибкие правила sampling
 - Использование Tag-ов и Attributes
 - Мониторинг с alert-ами
 - Оптимизация span-ов
1. Сбор данных только там, где это важно
 2. Легкая фильтрация и поиск информации в дашбордах
 3. Быстрое реагирование на проблемы производительности
 4. Минимизация накладных расходов и лишних span-ов

Подсказка: Думайте, как сделать систему информативной, но лёгкой.

- **Гибкие правила sampling** — Сбор данных только там, где это важно
- **Использование Tag-ов и Attributes** — Легкая фильтрация и поиск информации в дашбордах
- **Мониторинг с alert-ами** — Быстрое реагирование на проблемы производительности
- **Оптимизация span-ов** — Минимизация накладных расходов и лишних span-ов

Шаг 07:

Миссия 7

Сопоставьте рекомендации по OTEL с их эффектом:

Ответ:

- Отправка данных пакетами
 - Использование асинхронных экспортеров
 - Сохранение контекста при вызовах
 - Фокус на критические span-ы
1. Снижение нагрузки на сеть и collector
 2. Не блокировать выполнение приложения
 3. Корректная трассировка распределённых запросов
 4. Сбор только важной информации

Подсказка: Думайте о связи между практикой и тем, что она улучшает.

- **Отправка данных пакетами** — Снижение нагрузки на сеть и collector
- **Использование асинхронных экспортеров** — Не блокировать выполнение приложения
- **Сохранение контекста при вызовах** — Корректная трассировка распределённых запросов
- **Фокус на критические span-ы** — Сбор только важной информации

Шаг 08:

Миссия 8

Расположите шаги внедрения OTEL в high-load систему в логическом порядке:

1. Определение критических сервисов и операций
2. Настройка sampling и batch export
3. Интеграция асинхронных экспортеров
4. Сбор метрик и трасс с сохранением контекста
5. Мониторинг и alert-ы на основе данных

Подсказка: Сначала определяем, что важно, затем настраиваем сбор, интегрируем и наблюдаем.

Ответ:

- Определение критических сервисов и операций
- Настройка sampling и batch export
- Интеграция асинхронных экспортеров
- Сбор метрик и трасс с сохранением контекста
- Мониторинг и alert-ы на основе данных

Урок 2 - Типичные ошибки и как их избежать

Шаг 01:

Миссия 1

Какая из этих практик является типичной ошибкой при внедрении OpenTelemetry?

Подсказка: Подумайте, что может сильно нагрузить систему.

Ответ:

- **Собирать все span-ы синхронно без sampling**

- Использовать batch export и sampling
- Сохранять контекст трасс при межсервисных вызовах

Шаг 02:

Миссия 2

Что из перечисленного может привести к потере важных данных в OTEL?

Подсказка: Думайте о том, когда данные могут не попасть в collector.

Ответ:

- **Отключение трассировки или sampling без учета критических span-ов**
- Использование batch export для снижения нагрузки
- Асинхронная отправка данных

Шаг 03:

Миссия 3

Какая ошибка чаще всего встречается при распределённой трассировке?

Подсказка: Подумайте, что происходит при вызовах между сервисами.

Ответ:

- **Потеря контекста span-ов при межсервисных вызовах**
- Оптимизация span-ов для снижения нагрузки
- Использование sampling для критических запросов

Шаг 04:

Миссия 4

Что может вызвать деградацию производительности приложения при OTEL?

Подсказка: Подумайте о накладных расходах на сбор данных.

Ответ:

- **Сбор span-ов по одному в синхронном режиме**
- Использование batch export
- Асинхронная отправка данных

Шаг 05:

Миссия 5

Сопоставьте типичные ошибки OTEL с их последствиями:

Ответ:

- Сбор всех span-ов без sampling
 - Потеря контекста span-ов
 - Синхронная отправка данных
 - Игнорирование критических span-ов
1. Сильная нагрузка на приложение и collector
 2. Невозможность корректной трассировки распределённых запросов
 3. Замедление работы приложения
 4. Пропуск важных данных и снижение информативности мониторинга

Подсказка: Думайте, какая ошибка ведёт к потере данных, а какая — к нагрузке.

- **Сбор всех span-ов без sampling** — Сильная нагрузка на приложение и collector
- **Потеря контекста span-ов** — Невозможность корректной трассировки распределённых запросов
- **Синхронная отправка данных** — Замедление работы приложения
- **Игнорирование критических span-ов** — Пропуск важных данных и снижение информативности мониторинга

🔧 Шаг 06:

Миссия 6

Сопоставьте ошибки и лучшие практики для их предотвращения:

Ответ:

- Сбор span-ов без sampling
- Потеря контекста при межсервисных вызовах
- Синхронная отправка данных
- Отсутствие alert-ов

1. Использовать sampling и batch export
2. Сохранять контекст трасс между сервисами
3. Применять асинхронные экспортеры
4. Настроить мониторинг с alert-ами

Подсказка: Каждой ошибке соответствует своя профилактика.

- **Сбор span-ов без sampling** — Использовать sampling и batch export
- **Потеря контекста при межсервисных вызовах** — Сохранять контекст трасс между сервисами
- **Синхронная отправка данных** — Применять асинхронные экспортеры
- **Отсутствие alert-ов** — Настроить мониторинг с alert-ами

🔧 Шаг 07:

Миссия 7

Сопоставьте ошибки OTEL с их проявлениями в системе:

Ответ:

- Пропуск критических span-ов
 - Перегрузка collector
 - Некорректная трассировка
 - Замедление приложения
1. Снижается точность мониторинга и анализа
 2. Падает скорость обработки данных
 3. Невозможно отследить сквозные запросы
 4. Пользовательский опыт ухудшается

Подсказка: Подумайте, какое проявление связано с какой ошибкой.

- **Пропуск критических span-ов** — Снижается точность мониторинга и анализа
- **Перегрузка collector** — Падает скорость обработки данных
- **Некорректная трассировка** — Невозможно отследить сквозные запросы
- **Замедление приложения** — Пользовательский опыт ухудшается

Шаг 08:

Миссия 8

Расположите шаги предотвращения типичных ошибок OTEL в правильном порядке:

1. Определение критически важных операций и span-ов
2. Настройка sampling и batch export
3. Сохранение контекста распределённых вызовов
4. Использование асинхронной отправки данных
5. Настройка мониторинга и alert-ов

Подсказка: Сначала выявляем, что важно, потом настраиваем сбор, интегрируем и наблюдаем.

Ответ:

- Определение критически важных операций и span-ов
- Настройка sampling и batch export
- Сохранение контекста распределённых вызовов
- Использование асинхронной отправки данных
- Настройка мониторинга и alert-ов

Шаг 09:

Миссия 9

Расположите шаги анализа последствий ошибок OTEL:

1. Сбор метрик и span-ов
2. Выявление узких мест и пропущенных span-ов
3. Анализ нагрузки на collector и приложение
4. Оптимизация конфигурации OTEL и sampling
5. Тестирование изменений и мониторинг

Подсказка: Сначала собираем и анализируем данные, потом корректируем и проверяем.

Ответ:

- Сбор метрик и span-ов
- Выявление узких мест и пропущенных span-ов
- Анализ нагрузки на collector и приложение
- Оптимизация конфигурации OTEL и sampling
- Тестирование изменений и мониторинг

Урок 3 - Настройка продакшн-ready мониторинга

Шаг 01:

Миссия 1

Что является основным элементом продакшн-ready мониторинга в OpenTelemetry?

Подсказка: Подумайте, какие данные помогают измерять состояние и производительность системы.

Ответ:

- **Метрики, трассы и логи**
- Только лог-файлы
- Только конфигурационные файлы

Шаг 02:

Миссия 2

Почему важно настроить структурированные логи в продакшн-системах?

Подсказка: Подумайте, как облегчить поиск и фильтрацию событий при высокой нагрузке.

Ответ:

- **Чтобы легко фильтровать и анализировать события**
- Чтобы увеличить объём данных
- Чтобы уменьшить производительность мониторинга

Шаг 03:

Миссия 3

Что из перечисленного обеспечивает сквозную трассировку запросов через микросервисы?

Подсказка: Подумайте о следе, который проходит через весь путь запроса.

Ответ:

- **Распределённые трассы (distributed traces)**
- Локальные логи
- Простые счётчики метрик

Шаг 04:

Миссия 4

Зачем использовать атрибуты и теги в метриках и трассах?

Подсказка: Подумайте, как можно анализировать данные по различным критериям.

Ответ:

- **Чтобы группировать и фильтровать события**
- Чтобы замедлить сбор данных
- Чтобы игнорировать ошибки сервисов

Шаг 05:

Миссия 5

Сопоставьте элементы мониторинга OTEL с их задачами:

Ответ:

- Метрики
 - Логи
 - Распределённые трассы
 - Атрибуты/теги
1. Измерение состояния и производительности сервисов
 2. Отслеживание ошибок и событий
 3. Связывание операций между сервисами
 4. Фильтрация и группировка данных

Подсказка: Каждый элемент решает отдельную задачу: измерение, анализ или трассировку.

- **Метрики** — Измерение состояния и производительности сервисов
- **Логи** — Отслеживание ошибок и событий
- **Распределённые трассы** — Связывание операций между сервисами

- **Атрибуты/теги** — Фильтрация и группировка данных

Шаг 06:

Миссия 6

Сопоставьте практики логирования и трассировки с их эффектами:

Ответ:

- Structured Logging
 - Context Propagation
 - Sampling
 - Alerting
1. Упрощает фильтрацию и поиск событий
 2. Сохраняет контекст между сервисами
 3. Снижает нагрузку на сбор данных
 4. Позволяет быстро реагировать на проблемы

Подсказка: Думайте о читаемости, нагрузке и скорости реакции.

- **Structured Logging** — Упрощает фильтрацию и поиск событий
- **Context Propagation** — Сохраняет контекст между сервисами
- **Sampling** — Снижает нагрузку на сбор данных
- **Alerting** — Позволяет быстро реагировать на проблемы

Шаг 07:

Миссия 7

Сопоставьте элементы распределённой системы с их функцией:

Ответ:

- Load Balancer
 - Service Mesh
 - Message Queue
 - Database Sharding
1. Равномерное распределение нагрузки
 2. Маршрутизация трафика и наблюдаемость
 3. Асинхронная обработка сообщений
 4. Масштабирование данных базы

Подсказка: Думайте, как система справляется с масштабом и взаимодействием сервисов.

- **Load Balancer** — Равномерное распределение нагрузки
- **Service Mesh** — Маршрутизация трафика и наблюдаемость
- **Message Queue** — Асинхронная обработка сообщений
- **Database Sharding** — Масштабирование данных базы

Шаг 08:

Миссия 8

Расположите шаги внедрения продакшн-ready мониторинга в правильной последовательности:

1. Определение критических сервисов и метрик
2. Настройка логирования и структурированных логов

3. Настройка трассировки и propagation контекста
4. Настройка sampling и batch export
5. Мониторинг, alert-ы и визуализация

Подсказка: Сначала выбираем, что мониторить, затем настраиваем сбор данных и оповещения.

Ответ:

- Определение критических сервисов и метрик
- Настройка логирования и структурированных логов
- Настройка трассировки и propagation контекста
- Настройка sampling и batch export
- Мониторинг, alert-ы и визуализация

Шаг 09:

Миссия 9

Расположите этапы анализа производительности распределённой системы с OTEL:

1. Сбор метрик, логов и трасс
2. Идентификация узких мест
3. Оптимизация кода и конфигурации сервисов
4. Проверка эффективности изменений

Подсказка: Подумайте: сначала собираем данные, потом анализируем, затем улучшаем, и проверяем результат.

Ответ:

- Сбор метрик, логов и трасс
- Идентификация узких мест
- Оптимизация кода и конфигурации сервисов
- Проверка эффективности изменений

Шаг 10:

Миссия 10

Расположите шаги настройки продакшн-ready трассировки для микросервисной системы:

1. Инструментирование кода с OTEL SDK
2. Включение контекстной передачи между сервисами
3. Настройка экспорта трасс в систему наблюдения
4. Проверка визуализации и анализа трасс

Подсказка: Сначала готовим код, потом передаём контекст, экспортируем данные и проверяем визуализацию.

Ответ:

- Инструментирование кода с OTEL SDK
- Включение контекстной передачи между сервисами
- Настройка экспорта трасс в систему наблюдения
- Проверка визуализации и анализа трасс

Шаг 11:

Вопрос 11

Что является принципом "чистого кода" при рефакторинге?

Подсказка: Уменьшение дублирования — ключ к чистоте кода.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Добавление длинных комментариев
- Смешивание логики и интерфейса
- Использование непонятных сокращений

Шаг 12:

Вопрос: Расположите шаги рефакторинга в правильном порядке:

1. Запуск тестов и проверка, что всё работает, как раньше
2. Выбор проблемного участка кода
3. Внесение улучшений (читаемость, удаление дубликатов и т.д.)
4. Поддержание функциональности без изменений

 **Подсказка:**

Подумайте:

Ответ:

- Сначала нужно определить, где именно код требует улучшений.
- Потом выполняются изменения.
- Важно помнить, что поведение программы не должно измениться.
- И только в конце нужно убедиться с помощью тестов, что всё осталось как раньше.
- Выбор проблемного участка кода
- Внесение улучшений (читаемость, удаление дубликатов и т.д.)
- Поддержание функциональности без изменений
- Запуск тестов и проверка, что всё работает, как раньше

Шаг 13:

 **Задача: Сопоставление понятий рефакторинга**

Формулировка: Сопоставьте понятие с его правильным объяснением.

Понятия:

Ответ:

- Рефакторинг
- Цель рефакторинга
- Проблема "дублирование кода"
- Проблема "плохие названия"

Определения:

1. Изменение внутренней структуры кода без изменения внешнего поведения.
2. Сделать код более понятным, удобным для поддержки и расширения.
3. Когда один и тот же фрагмент программы повторяется много раз.
4. Когда имена переменных или функций не отражают их смысл.

 **Подсказка:**

Подумайте, зачем нужен рефакторинг: он не добавляет новые функции, а улучшает сам код. Обратите внимание на типичные проблемы — они часто связаны либо с повторением, либо с непонятными именами.

- **Рефакторинг** — Изменение внутренней структуры кода без изменения внешнего поведения.
- **Цель рефакторинга** — Сделать код более понятным, удобным для поддержки и расширения.
- **Проблема "дублирование кода"** — Когда один и тот же фрагмент программы повторяется много раз.
- **Проблема "плохие названия"** — Когда имена переменных или функций не отражают их смысл.

Урок 4 - Дальнейшее изучение метрики, логирование, распределённые систем

Шаг 01:

Миссия 1

Что из перечисленного является основой метрик в OpenTelemetry?

Подсказка: Подумайте о числовых данных, которые помогают измерять производительность.

Ответ:

- **Счётчики, гистограммы и измерения времени**
- Только лог-файлы
- Только распределённые трассы

Шаг 02:

Миссия 2

Почему логирование важно в высоконагруженных системах с OTEL?

Подсказка: Подумайте о том, как понять поведение системы при ошибках.

Ответ:

- **Позволяет отследить ошибки и события в системе**
- Заменяет метрики и трассировки
- Не нужен в продакшн-системах

Шаг 03:

Миссия 3

Что из перечисленного помогает связывать события между разными сервисами?

Подсказка: Подумайте о «следе», который идёт через всю систему.

Ответ:

- **Распределённые трассы (distributed traces)**
- Локальные логи
- Счётчики метрик только одного сервиса

Шаг 04:

Миссия 4

Почему важно использовать атрибуты и теги в метриках и трассах?

Подсказка: Подумайте о том, как фильтровать и группировать данные.

Ответ:

- **Чтобы можно было фильтровать, группировать и искать события**

- Чтобы увеличить размер данных без пользы
- Чтобы уменьшить производительность мониторинга

Шаг 05:

Миссия 5

Сопоставьте элементы мониторинга OTEL с их назначением:

Ответ:

- Метрики
 - Логи
 - Распределённые трассы
 - Атрибуты/теги
1. Измерение производительности и состояния сервисов
 2. Отслеживание ошибок и событий
 3. Связывание операций между сервисами
 4. Фильтрация и группировка данных

Подсказка: Каждый элемент решает отдельную задачу: измерение, отладка или анализ.

- **Метрики** — Измерение производительности и состояния сервисов
- **Логи** — Отслеживание ошибок и событий
- **Распределённые трассы** — Связывание операций между сервисами
- **Атрибуты/теги** — Фильтрация и группировка данных

Шаг 06:

Миссия 6

Сопоставьте подходы к логированию и трассировке с их эффектами:

Ответ:

- Structured Logging
 - Context Propagation
 - Sampling
 - Alerting
1. Упрощает фильтрацию и поиск событий
 2. Сохраняет сквозной контекст между сервисами
 3. Снижает нагрузку на систему при сборе данных
 4. Позволяет быстро реагировать на проблемы

Подсказка: Подумайте, какая практика улучшает читаемость, какая снижает нагрузку.

- **Structured Logging** — Упрощает фильтрацию и поиск событий
- **Context Propagation** — Сохраняет сквозной контекст между сервисами
- **Sampling** — Снижает нагрузку на систему при сборе данных
- **Alerting** — Позволяет быстро реагировать на проблемы

Шаг 07:

Миссия 7

Сопоставьте элементы распределённых систем с их ролью:

Ответ:

- Load Balancer
 - Service Mesh
 - Message Queue
 - Database Sharding
1. Равномерное распределение нагрузки между сервисами
 2. Обеспечение наблюдаемости и маршрутизации трафика
 3. Очередь сообщений для асинхронной обработки
 4. Разделение данных для масштабирования базы

Подсказка: Думайте о том, как системы справляются с масштабом и взаимодействием.

- **Load Balancer** — Равномерное распределение нагрузки между сервисами
- **Service Mesh** — Обеспечение наблюдаемости и маршрутизации трафика
- **Message Queue** — Очередь сообщений для асинхронной обработки
- **Database Sharding** — Разделение данных для масштабирования базы

 Шаг 08:

Миссия 8

Расположите шаги внедрения продвинутого мониторинга OTEL в логической последовательности:

1. Определение критических сервисов и метрик
2. Настройка логирования и структурированных логов
3. Настройка трассировок и propagation контекста
4. Настройка sampling и batch export
5. Мониторинг, alert-ы и визуализация

Подсказка: Сначала определяем важное, потом настраиваем сбор, затем анализ и уведомления.

Ответ:

- Определение критических сервисов и метрик
- Настройка логирования и структурированных логов
- Настройка трассировок и propagation контекста
- Настройка sampling и batch export
- Мониторинг, alert-ы и визуализация